



國立中山大學資訊工程學系

碩士論文

Department of Computer Science and Engineering

National Sun Yat-sen University

Master's Thesis

多域SDN網路中利益導向資料流的協同路由管理

Cooperative Route Management for Profit-Oriented Flows  
in Multi-domain SDN Networks

研究生：周孟佑

Meng-Yu Chou

指導教授：王友群 博士

Dr. You-Chiun Wang

中華民國114年9月

September 2025



國立中山大學資訊工程學系

碩士論文

Department of Computer Science and Engineering

National Sun Yat-sen University

Master's Thesis

多域 SDN 網路中利益導向資料流的協同路由管理

Cooperative Route Management for Profit-Oriented Flows  
in Multi-domain SDN Networks

研究生：周孟佑

Meng-Yu Chou

指導教授：王友群 博士

Dr. You-Chiun Wang

中華民國 114 年 9 月

September 2025

# 論文審定書

國立中山大學研究生學位論文審定書

本校資訊工程學系碩士班

研究生周孟佑（學號：M123040011）所提論文

多域SDN網路中利益導向資料流的協同路由管理  
Cooperative route management for profit-oriented flows in multi-domain  
SDN networks

於中華民國 114 年 8 月 21 日經本委員會審查並舉行口試，符合碩士學位論文標準。

學位考試委員簽章：

召集人 余亞儒 余亞儒

委員 王友群 王友群

委員 劉炳宏 劉炳宏

委員 林俊宏 林俊宏

委員 \_\_\_\_\_

委員 \_\_\_\_\_

指導教授(王友群) 王友群 (簽名)

# 論文公開授權書

## 誌謝

首先，我要衷心感謝我的指導教授 王友群 博士。老師在研究過程中不僅提供了許多寶貴的建議與研究方向，更在學術態度上給了我很大的啟發。這段時間的指導讓我逐漸培養出獨立思考與解決問題的能力，也讓我在學術與人生道路上都有了成長。感謝老師在這兩年中的耐心指導與照顧，讓我能夠一步步走到今天。

同時，也要感謝我的口試委員 林俊宏 教授、劉炳宏 教授以及 余亞儒 教授。謝謝各位在論文口試時給予我中肯的建議與不同角度的看法，這些回饋使我的研究更加完整，也讓我更深入地思考自己的論文主題。

在研究所生活中，我要感謝已經畢業的學長姊們，讓我能更快地適應環境，並在課業或研究上遇到困難時給予我協助與指引。同時，特別感謝實驗室的夥伴們——柏榕、志昇、沛錦、玉琪、玉文、鎧均、秉澄。謝謝你們一路上的陪伴與支持，無論是在研究上的討論，還是生活中的鼓勵，都讓我能更堅定地走下去。也要感謝學弟們，你們在我遇到瓶頸時的鼓勵與交流，常常讓我理清思緒，找到新的解決方法。

最後，我要最深深地感謝我的家人。在我遇到挫折、感到低落的時候，你們總是給予我最大的支持與力量，成為我最堅強的後盾。因為有你們無條件的陪伴與鼓勵，我才能心無旁騖地完成學業。

## 摘要

在多域 SDN 網路的環境中，隨著裝置的激增與大數據應用的普及，現有網路架構面臨流量調控不靈活、規則不統一以及資源利用效率低落等挑戰，過往方法在跨網域流量管理上主要關注在負載平衡與封包遺失率，然而，這些方法未能充分考量資料流對於系統的經濟價值，導致部分高價值資料流可能會因在資源分配不理想，而造成網路整體效能與服務品質的降低。

本研究提出一種考慮資料流價值並且採取利益導向的跨網域資料流協同路由管理機制，根據資料流的價值來動態調整頻寬以及路由選擇，以確保不同類型的流量皆能獲得適當資源分配，在此機制當中，當特定網域因流量過載而造成某些鏈路發生壅塞時，可將部分流量導入其他網域來進行協同傳輸，同時，為避免協助網域自身的流量受到過度影響，我們會使用 Nash Bargaining 啟發的方式來判斷要借給其他網域的頻寬，並依照鏈路情況對自身網域的資料流進行限速，以確保價值的產出，此外，在沒有協助其他網域情況時，倘若網路發生壅塞時，會先尋找替代路徑，而若無法找到替代路徑以及向其他網域借路，則會進行流量限速，根據壅塞鏈路的資料流頻寬需求與價值，來動態優化資源使用策略，以保障系統的總價值產出，透過 Mininet 模擬驗證，本論文所提出的機制能夠有效提升多域 SDN 網路的吞吐量，並提升系統價值的產出，其不僅增強，跨網域流量管理的靈活性與資源利用效率，也為未來 SDN 網路的流量調度提供一種利益導向的解決方案。

**關鍵詞：**協同式傳輸、負載平衡、多域網路、OpenFlow、軟體定義網路。

# Abstract

In multi-domain *Software-Defined Networking* (SDN) environments, the rapid growth of devices and the widespread adoption of big-data applications have imposed significant challenges to existing network architectures, including inflexible traffic control, lack of unified policies, and inefficient resource utilization. Existing approaches to cross-domain traffic management focus primarily on load balance or packet loss. However, these methods disregard the economic value of individual flows to the system, resulting in suboptimal resource allocation that may degrade network performance and quality of service, particularly for high-value flows.

This thesis proposes a profit-oriented cross-domain cooperative method to dynamically adjust bandwidth allocation and routing decisions based on the profit associated with each flow. When a domain experiences congestion due to traffic overload, the selected flow can be offloaded to neighboring domains for cooperative transmission. To prevent excessive impact on the assisting domain's own traffic, we propose a Nash Bargaining inspired scheme to determine the amount of bandwidth to lend and applies rate-limiting policies—via OpenFlow's meter table—on local flows to preserve the overall system profit. Furthermore, in the absence of external assistance, the system first attempts to reroute traffic within the local domain. If no alternative paths are available, it enforces rate control based on the bandwidth demands and profit values of congested flows in order to improve resource usage.

Through simulations performed using Mininet, we validate the effectiveness of our proposed method in improving the total throughput and profit output of multi-domain SDN networks. Our method not only enhances the flexibility of cross-domain traffic coordination and resource efficiency, but also introduces a profit-oriented solution for future SDN traffic management.

**Keywords:** cooperative management, load balancing, multi-domain networks, OpenFlow, software-defined networking

# 目錄

|                                  |      |
|----------------------------------|------|
| 論文審定書 .....                      | i    |
| 論文公開授權書 .....                    | ii   |
| 誌謝 .....                         | iii  |
| 摘要 .....                         | iv   |
| Abstract .....                   | v    |
| 目錄 .....                         | vi   |
| 圖次 .....                         | viii |
| 表次 .....                         | x    |
| 第一章 緒論 .....                     | 1    |
| 第二章 研究背景 .....                   | 3    |
| 2.1 SDN 網路架構 .....               | 3    |
| 2.2 OpenFlow 協定 .....            | 4    |
| 2.2.1 Flow Table .....           | 5    |
| 2.2.2 Group Table .....          | 7    |
| 2.2.3 Meter Table .....          | 10   |
| 2.3 常見的 SDN Controller 介紹 .....  | 11   |
| 第三章 相關文獻探討 .....                 | 15   |
| 3.1 控制器間的通訊機制 .....              | 16   |
| 3.2 控制器負載平衡 .....                | 18   |
| 3.3 網路鏈路負載平衡 .....               | 19   |
| 3.4 討論 .....                     | 20   |
| 第四章 問題定義 .....                   | 22   |
| 第五章 研究方法 .....                   | 26   |
| 5.1 網路拓撲初始化與跨網域溝通 .....          | 28   |
| 5.2 流量定期監控 .....                 | 31   |
| 5.3 壅塞與跨域負載平衡 .....              | 36   |
| 5.3.1 同網域的負載平衡 .....             | 37   |
| 5.3.2 跨網域負載平衡 .....              | 38   |
| 5.4 Nash Bargaining 模組 .....     | 40   |
| 5.5 Profit Maximization 模組 ..... | 43   |
| 第六章 效能評估 .....                   | 48   |
| 6.1 實驗環境與工具 .....                | 48   |
| 6.2 網路拓撲與流量配置 .....              | 49   |
| 6.3 實驗 1：整體效能評估 .....            | 50   |
| 6.3.1 Throughput 分析 .....        | 53   |



|                                      |    |
|--------------------------------------|----|
| 6.3.2 獲取 Profit 分析 .....             | 57 |
| 6.3.3 封包丟失率分析 .....                  | 61 |
| 6.4 實驗 2：不同 Profit 與頻寬需求的相關性分析 ..... | 62 |
| 6.4.1 實驗 2 結果分析 .....                | 64 |
| 第七章 結論與未來展望 .....                    | 69 |
| 7.1 結論 .....                         | 69 |
| 7.2 未來展望 .....                       | 69 |
| 參考文獻 .....                           | 70 |
| Appendix .....                       | 75 |

## 圖次

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| 圖 2-1：SDN 架構之示意圖 .....                                                                       | 4  |
| 圖 2-2：OpenFlow 交換機之內部元件示意圖 .....                                                             | 5  |
| 圖 2-3：OpenFlow Pipeline 技術用於管理 Flow Table .....                                              | 6  |
| 圖 2-4：Flow_Mod 訊息的結構 .....                                                                   | 6  |
| 圖 2-5：Flow entry 結構 .....                                                                    | 6  |
| 圖 2-6：Group_Mod 封包之結構 .....                                                                  | 8  |
| 圖 2-7：Group Entry 封包之結構 .....                                                                | 8  |
| 圖 2-8：Group identifier 範例，上表為有 Flow entry 的 Flow table，下表則為有 Group entry 的 Group table ..... | 9  |
| 圖 2-9：Meter_Mod 的封包結構 .....                                                                  | 10 |
| 圖 2-10：Meter entry 之結構 .....                                                                 | 10 |
| 圖 2-11：Meter table 範例，上表為有 Flow entry 的 Flow table，下表為有 Meter enter 的 Meter table .....      | 11 |
| 圖 2-12：RYU 運作架構 .....                                                                        | 13 |
| 圖 3-1：集中式 SDN 網路管理拓譜的範例 .....                                                                | 16 |
| 圖 3-2：平面式的分散式 SDN 網路管理拓譜範例 .....                                                             | 17 |
| 圖 3-3：階層式的分散式 SDN 網路管理拓譜範例 .....                                                             | 17 |
| 圖 4-1：在具協同合作的多網域下的封包繞率之範例 .....                                                              | 22 |
| 圖 5-1：本系統之流程圖 .....                                                                          | 28 |
| 圖 5-2：多網域 SDN 網路的拓譜範例 .....                                                                  | 29 |
| 圖 5-3：內部負載平衡示意圖 .....                                                                        | 38 |
| 圖 5-4：跨網域協助傳輸之範例 .....                                                                       | 39 |
| 圖 6-1：實驗所使用的網路拓樸圖 .....                                                                      | 49 |
| 圖 6-2：實驗 1 Case 1 資料流存續時間 .....                                                              | 51 |
| 圖 6-3：實驗 1 Case 2 資料流存續時間 .....                                                              | 52 |
| 圖 6-4：實驗 1 Case 3 資料流存續時間 .....                                                              | 52 |
| 圖 6-5：實驗 1 Case 4 資料流存續時間 .....                                                              | 52 |
| 圖 6-6：實驗 1 Case 1 之整體系統吞吐量 .....                                                             | 55 |
| 圖 6-7：實驗 1 Case 2 之整體系統吞吐量 .....                                                             | 55 |
| 圖 6-8：實驗 1 Case 3 之整體系統吞吐量 .....                                                             | 56 |
| 圖 6-9：實驗 1 Case 4 之整體系統吞吐量 .....                                                             | 56 |
| 圖 6-10：實驗 1 Case 1 所獲取的 Profit .....                                                         | 58 |
| 圖 6-11：實驗 1 Case 2 所獲取的 Profit .....                                                         | 59 |
| 圖 6-12：實驗 1 Case 3 所獲取的 Profit .....                                                         | 59 |

|                                     |    |
|-------------------------------------|----|
| 圖 6-13：實驗 1 Case 4 所獲取的 Profit..... | 59 |
| 圖 6-14：實驗 2 Case 1 資料流存續時間 .....    | 63 |
| 圖 6-15：實驗 2 Case 2 資料流存續時間 .....    | 63 |
| 圖 6-16：實驗 2 Case 3 資料流存續時間 .....    | 63 |
| 圖 6-17：實驗 2 Case 1 各資料流封包丟失率 .....  | 64 |
| 圖 6-18：實驗 2 Case 2 各資料流封包丟失率 .....  | 65 |
| 圖 6-19：實驗 2 Case 3 各資料流封包丟失率 .....  | 65 |
| 圖 6-20：實驗 2 各情境之總 Profit 獲取.....    | 67 |

## 表次

|                                                    |    |
|----------------------------------------------------|----|
| 表 4-1：本研究使用的參數.....                                | 25 |
| 表 5-1：控制器訊息之參數表.....                               | 30 |
| 表 5-2：交換機之狀態表.....                                 | 31 |
| 表 5-3：主機交換機位置表範例 (以 Domain 2 為例) .....             | 32 |
| 表 5-4：資料流路徑紀錄表範例(三個網域各自的表).....                    | 33 |
| 表 5-5：網域 2 資料流交換機情報表範例.....                        | 34 |
| 表 5-6：網域 2 的流量紀錄與封包丟失率之計算範例.....                   | 35 |
| 表 5-7：模組中各參數所代表的意義.....                            | 43 |
| 表 5-8：Profit Maximization 模組之參數表 .....             | 45 |
| 表 5-9：頻寬要求與 Profit 不相關.....                        | 47 |
| 表 5-10：頻寬要求與 Profit 正相關.....                       | 47 |
| 表 5-11：頻寬要求與 Profit 負相關.....                       | 47 |
| 表 6-1：模擬工具與其使用版本.....                              | 48 |
| 表 6-2：實驗 1 的資料流設定.....                             | 53 |
| 表 6-3：實驗 1 封包丟失率與 Profit 獲取.....                   | 60 |
| 表 6-4：實驗 1 各情境中 Profit 獲取以及各方法與 PNCFM 結果之比較.....   | 60 |
| 表 6-5：實驗 2 各情境的資料流資訊.....                          | 64 |
| 表 6-6：實驗 2 各情境下之封包丟失率與 Profit 獲取.....              | 66 |
| 表 6-7：實驗 2 各情境下總 Profit 獲取結果與 PNCFM 比其他方法改善幅度..... | 66 |

# 第一章 緒論

隨著資訊技術的日新月異，網路也遇到不同的難題，像是需要應對大量增加的移動裝置、大數據的傳輸、大量存取雲端系統；而傳統網路架構可能會遇到網路拓譜變複雜、難以管理、容易出現錯誤等問題。特別來講，傳統網路的資料層與控制層的高度耦合，使得更新傳統網路裝置與架構變得更為複雜，且進行細顆粒度的網路流量控制變為相當困難，也因此學者們提出軟體定義網路 (Software-defined networking，縮寫為 SDN) 的概念並於 2008 提出 OpenFlow 協定 [12]。

SDN 主要有兩個特點，第一點是將網路架構拆分為控制層(Control plane)與資料層(Data plane)，前者負責進行決策與管理，後者則依照控制層的指示進行資料的轉發或處理[3]；而第二點是將所有控制能力交由控制器(Controller)進行集中式的管理，讓控制器對規則進行設定與管理，而網路設備，諸如交換器(switch)等，只需依照規則去對封包進行指示的動作，其中，控制器與交換機透過 OpenFlow 等通訊協定來進行溝通，以實現網路流量的監管、規則的更新、以及其他網路相關設置，協助控制器妥善並全盤瞭解網路的狀況[3]。

一個採用 SDN 架構的系統可以即時更新規則，網路封包的路徑在特定情況下(例如：負載平衡考量、安全因素等)需要修改或是更新規則時可以進行修改，達成一個更具彈性的網路，跟傳統網路相比更可大幅節省維護以及調控的成本，其中包含較為特殊的方式，例如跨網域異質流合作的方法 [3]，也就是在當前網域壅塞難以負擔新的流量的情況下，可將部分流量導出該網域，從不同閘道 (Gateway) 回到原網域的方式，以此方式繞開網域中壅塞的鏈路(Link)，這個方式的好處是可以提升整體的吞吐量，降低原網域的負載，同時顧及不要讓外網流量影響自身網域的流量導致餓死(Starvation)情況發生。

此外，在特殊場景下，網路管理者會希望考慮資料流本身的價值[4]，其中，高價值的資料流會得到較高的頻寬保障，從而幫助網域獲得更高的經濟價值，本研究認為這樣的設定是文獻[3]並未考量的議題，並從中提出以下貢獻：

1. 類似於文獻[3]，本論文希望善用 SDN 特性，使不同網域間可以在發生壅塞時互相借用鏈路以傳送封包。
2. 網域協作的步驟中，協助網域是否進行協助，以及要提供多少協助可以有個較嚴謹的判斷機制，文獻[3]中採用優先級來分配異質流的頻寬佔比，但這方式並沒有考量資料流的價值，因此本研究設計啟發於 Nash Bargaining Solution 的策略，以計算協助網域借用路徑的流量所可分配到的頻寬。
3. 當網域發生壅塞時，本研究希望可以最大化每個網域可以產生的價值，達成整體價值的最大化，因此要建立對於資料流頻寬管理的機制，對部分或是全部流量進行細緻化的限速。
4. 最大化價值產出的同時，也應避免資料流沒有分配到頻寬而造成餓死 (Starvation) 的情況發生。
5. 本研究藉由 Mininet 模擬驗證本論文所開發的「價值談判協同式資料流管理」(Profit Negotiation collaborative flow management，簡稱為 PNCFM) 方法，以提升整體網域的價值產出。

本論文共分為七個章節，第二章介紹技術背景(包含 SDN、OpenFlow 協定等)，第三章則探討相關研究，而第四章闡明系統架構以及本文想要解決的問題，第五章論述我們所設計的 PNCFM 方法，第六章展示實驗數據並分析結果，第七章總結本論文並提出未來的研究議題。

## 第二章 研究背景

### 2.1 SDN 網路架構

相較於傳統的網路架構，SDN 將網路分為應用層(Application plane)、控制層(Control plane)、資料層(Data plane)，如圖 2-1 所示，並導入可編程(Programmable)的特性，以即時監控網路狀況並可機動調整封包傳送的規則。

應用層會負責執行運用 SDN 特性的應用程式，並可透過北向介面(Northbound interface)的 API (Application programming interface)與 SDN 控制器互動，以存取網路資源的相關訊息與狀態，應用服務供應商可以藉由 SDN 可編程的特性來達成他們的需求，例如：路由管理、權限控制、因應商業規定的客製化等[12]。

控制層則由 SDN 控制器(Controller)所組成，網路中所有的資料流規則(Flow rule)都由控制器所設定與更新，因此可進行集中化的網路資源管理，控制器並抽象化(Abstract)底層的硬體，這使得應用層可在不知道詳細的硬體資訊的情況下即可以執行應用程式，而網路服務供應商可以透過對控制器的程式進行修改來客製化所提供的服務。

資料層則有交換機與路由器等網路設備(Network devices)，但他們本身沒有控制功能，只會依照控制器設定好的網路規則來進行封包的傳送，控制層會透過南向介面(Southbound interface)與資料層進行溝通，其中常見的通訊協定為 OpenFlow，資料層還會將其接收到，但未建立規則的封包與設備的硬體資訊，透過南向介面傳送給控制器。

除了上述提及的北向介面與南向介面，控制層中還會有東西向介面(East-west bound interface)，該介面是多個控制器之間溝通的橋樑，可用於彼此間資訊交換[3][11]。

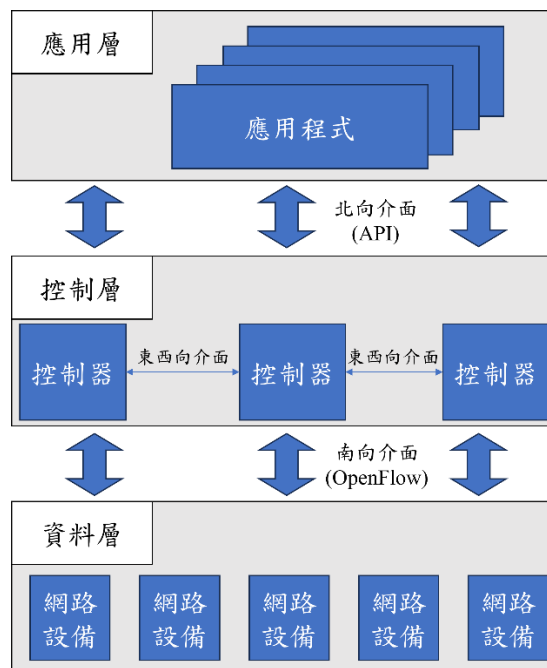


圖 2-1：SDN 架構之示意圖

## 2.2 OpenFlow 協定

Nick McKeown 團隊於 2008 年首度發表 OpenFlow 的白皮書，用以描述 OpenFlow 協定的運作方式[5]，OpenFlow 是網路通訊協定，提供對 SDN 資料層裝置的存取，它可運作於乙太網(Ethernet)的交換機上，其中，交換機內建資料流表(Flow-table)以及標準化的介面來新增或刪減流項目(Flow entry) [6]。控制器可以使用 OpenFlow 對交換機進行溝通，除此之外，OpenFlow 也可以運用於 VLAN 網路分隔、行動無線網路用戶、非 IP 網路、處理網路數據封包。OpenFlow 的發表也奠定後續 Open Networking Foundation (ONF) 於 2011 年成立，並進一步推動 OpenFlow 研發，其成員包含 Meta、Google、Microsoft 等各大廠。

圖 2-2 為 OpenFlow 交換機運作的示意圖，包含內部元件組成以及和各控制暨的互動，其支援三大功能：資料流表操作(Flow table operations)、安全通道(Secure channel)以及 OpenFlow 協議(OpenFlow protocol)，控制器會透過安全通道與交換機進行溝通，這些通道必須符合 OpenFlow 規定，以讓控制器可以管理與設定交換機。



接下來，我們會依序介紹 Flow table、Group table、Meter table，這些表格乃是 OpenFlow 中協助所有功能達成的重要元件。

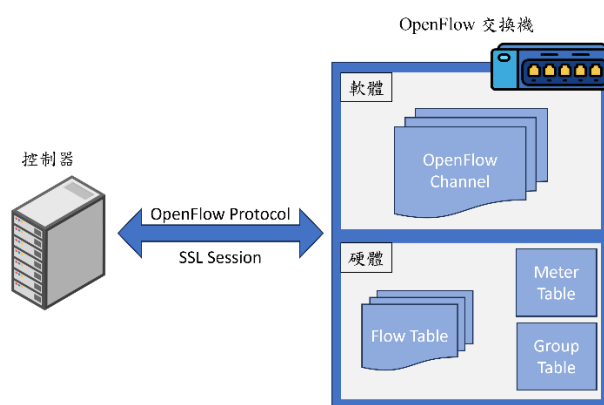


圖 2-2：OpenFlow 交換機之內部元件示意圖

## 2.2.1 Flow Table

OpenFlow 透過使用者定義或預設的流規則來匹配 (Match)和處理網路封包，這些規則構成 OpenFlow 的 Flow Table，而每個 Flow entry 則由三個元素組成，分別是標頭欄位 (Header)、動作 (Action)和統計數據(Stats)，封包會依據其標頭欄位進行匹配，然後根據 Flow entry 中的動作來進行處理。至於統計數據提供有關網路狀態的資訊，包括優先級、計數器、超時 (Timeout)、Cookie，以及其他欄位的內容，而於標頭中的每個欄位都可以用於模式匹配，因此網路運營商可以實現不同粒度 (granularity) 的流量控制[5]。

為了支援多種 OpenFlow 功能，Flow table 的大小可能會迅速增加，且一個交換機可能具有多個 Flow table，為了節省儲存空間，OpenFlow 標準採用 Pipeline 技術，即為一種類似於記憶體管理中多級頁表 (Multi-level page table) 的概念。封包到達交換機或是要更新 Flow entry 時，則會依照圖 2-3 描述的方式來查找 Flow entry，具體來說，交換機會依序查找每個 Flow table 並比對 Flow entry 規則，如果有符合規則的項目就會更新，而在處理封包或是更新規則時，如果沒有找到對應的 Flow entry，就會觸發 Table miss 事件。

在 Table miss 事件中，交換機會採用 Flow table 預設的 Flow entry  
 “priority=0,action=controller:6633”，其並無 Match field，Priority 為 0 (代表優先度最小)，而 Port 設定為預設值 6633，表示要將此封包轉送給 Controller 來取得下一步的指令。

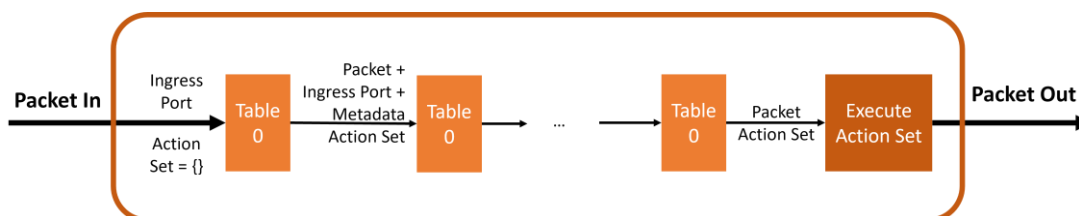


圖 2-3：OpenFlow Pipeline 技術用於管理 Flow Table

Flow table 內的標頭欄位、動作與統計數據代表著資料流的流向規則，控制器會發送 Flow\_Mod 訊息來更新它，而交換機在接收後會在其 Flow table 內建立一個相對應的 Flow entry，圖 2-4 描述 Flow\_Mod 訊息的結構。

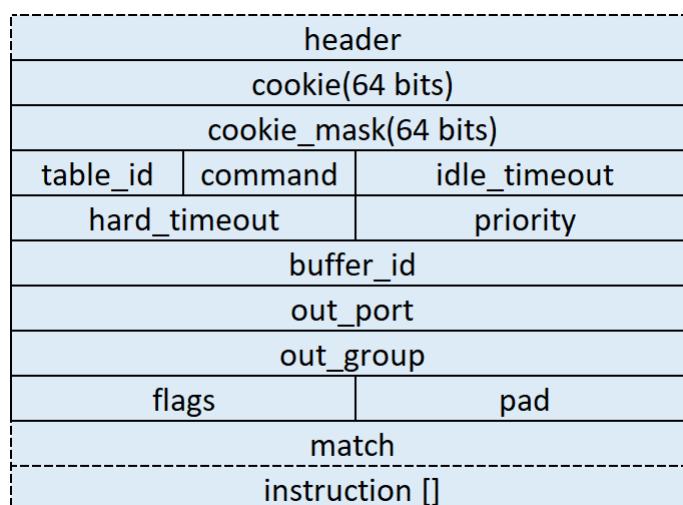


圖 2-4：Flow\_Mod 訊息的結構

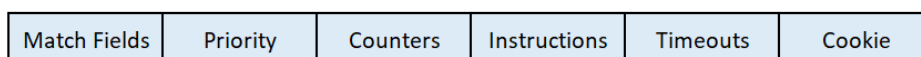


圖 2-5：Flow entry 結構

Flow entry 包含 Match fields、Priority、Counters、Instruction、Timeouts、Cookie 等六個欄位[5]，如圖 2-5 所示，以下逐一說明各欄位的功能：

1. Match fields：此欄位用於定義 Flow table 中 Flow entry 的匹配規則，乃是 Flow entries 的核心部分，匹配資訊包括交換機的 in\_port(輸入埠)、封包的 MAC 地址和 IP 地址，以及封包的標頭欄位(Header field)等。
2. Priority：當一個封包同時符合多個 Flow entry 的匹配條件時，就會根據此欄位來決定使用哪一條規則，其中優先權高的 Flow entry 將會被優先執行。
3. Counters：此欄位用於記錄 Flow entry 被觸發的次數，提供流量的統計資訊，來幫助監控網路資源使用情況。
4. Instruction：當封包匹配 Flow entry 的 Match fields 規則，並根據 Priority 確定執行該 Flow entry 後，交換機就會按照 Instruction 欄位中的指令對封包進行相關的處理，這些指令可以包括將封包轉發給控制器、執行基本操作諸如轉發(Forward)、泛洪(Flood)、丟棄(Drop)，或進行更精細的操作如更新 Meter table 或 Group table。
5. Timeouts：此欄位用於設定 Flow entry 的生命週期，包括 Hard timeout 以及 Idle timeout，其中 Hard timeout 是從該 Flow entry 建立開始計算，到達設定時間後 Flow entry 就會被刪除，至於 Idle timeout 是在 Flow entry 未被使用的情況下，從閒置開始計算並在時間達到時才進行刪除；而由於交換機記憶體有限，這項功能允許動態調整 Flow table 內容以提升資源的利用效率。
6. Cookie：此欄位用於記錄控制器對 Flow entry 的操作資訊，包含建立、修改或刪除相關資料的，方便控制器追蹤與管理 Flow table。

## 2.2.2 Group Table

在 OpenFlow 中，Group table 是一種用於支持更加複雜的封包處理行為之結構，與 Flow Table 搭配使用，它提供靈活的方式來實現多路徑傳輸(Multipath routing)、負載平衡 (Load balancing)、流量複製(Traffic mirroring)等功能，控制器

會發送 Group\_Mod 訊息(其封包結構請參考圖 2-6)來建立群組；Group table 可由一個或多個 Group entry 所組成，其結構如圖 2-7 所示，各欄位說明如下 [5]：

- Group identifier：其用於唯一標識每個 Group entry 的識別碼(ID)，為 32 bit 的 unsigned 整數。
- Group type：指定 Group entry 的類型，OpenFlow 提供四種不同的功能選項，包含 All、Select、Indirect、Fast failover，後續將對這些選項進行詳細說明。
- Counter：記錄該 Group entry 的使用次數，以便於統計和監控。
- Action buckets：由多個 Action bucket 所組成，每個 Bucket 包含需要執行的具體操作，並可包含其執行時的權重(Weight)，用於平衡多個 Bucket 的處理優先級。

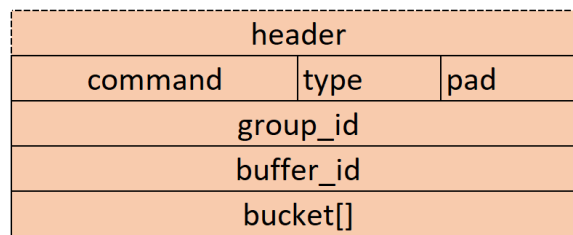


圖 2-6：Group\_Mod 封包之結構

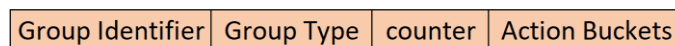


圖 2-7：Group Entry 封包之結構

Group type 分為 All、Select、Indirect、Fast failover 四類[4]，在此進行更詳細的解釋：

- 1) All：該類型執行 Group entry 中的所有 Action buckets，通常應用於群播 (Multicast)或是廣播(Broadcast)。當封包進入交換機時，會先複製該封包並執行相對應的操作，如果封包的輸出埠(Out port)與輸入埠(In port)相同，則該封包會被丟棄以避免產生無窮迴圈。
- 2) Select：此類型在封包進入時會執行其中一個 Action bucket，選擇的依據會是 Action bucket 的權重(Weight)，該權重決定被選中的機率，如果所選的 Action bucket 所對應的埠無法使用時，交換機會自動切換到其他可用的 Action bucket 來執行。

- 3) Indirect：該類型僅執行單一的 Action bucket，通常適用於僅有單一個 Action bucket 的場景，這是一種簡單且固定的行為模式。
- 4) Fast failover：該類型執行第一個可用的 Action bucket。如果所使用的埠發生故障，交換機會自動切換到下一個可用的 Action bucket，類似於 Select 的行為，但更專注於故障的恢復。

在這四種類型當中，All 和 Indirect 是交換機必須支援的功能，至於 Select 和 Fast failover 則是可選(Optional)功能，具體支援情況則取決於交換機的設計與能力支援[4]。

本論文中採用 Select 功能，以圖 2-8 為例進行說明。當封包進入交換機會先查詢 Flow table，尋找是否有匹配的規則，有匹配的 group\_id 則會到 Group table 內查詢對應的動作；具體來說，交換機會檢查 Flow table 中的 Match fields 欄位，倘若發現該封包來自 Port 1，且 eth\_type 為 0x800(即 IPv4 封包)，同時目標 IP 位址為 10.0.0.9，則會進一步查詢 Group table 中 Group identifier 為 3 的 Group entry，而根據該 Group entry 的資訊可知，Group type 為 Select，因此交換機會根據 Action buckets 的權重進行選擇，在此例中，有 20% 的機率會選擇 Port 3 傳送封包，有 80%的機率則會選擇 Port 1 傳送封包。

| Match                              | Priority   | Instruction     |
|------------------------------------|------------|-----------------|
| eth_type=0x800,ipv4_dst="10.0.0.9" | Priority=1 | Actions=Group:3 |

| Group Identifier | Group Type | Action Buckets                                                                                          |
|------------------|------------|---------------------------------------------------------------------------------------------------------|
| group_id = 3     | Select     | <div> <div>bucket=weight:80,actions=output:1,</div> <div>bucket=weight:20,actions=output:3</div> </div> |

圖 2-8：Group identifier 範例，上表為有 Flow entry 的 Flow table，下表則為有 Group entry 的 Group table

## 2.2.3 Meter Table

類似地，Meter table 是由多個 Meter entry 組成，用於定義每個資料流的流量 (即 Per-Flow Meter)，藉此，OpenFlow 可以實現多種 QoS 操作，例如速率限制 (Rate-limiting) 等，並且可以結合 Per-port 佇列來實現更複雜的 QoS 框架[5]。

Meter 的主要功能是計算分配給資料流的封包速率，並對這些封包的速率進行控制，Meter 是直接附加於 Flow entry，而非附加於埠(Port)的佇列。任何 Flow entry 都可以在其指令集中指定一個 Meter，該 Meter 將會對所有附加的 Flow entry 的封包聚合 (Aggregation) 進行計量與速率控制[5]。

同一個 Flow table 中可以使用多個 Meter，但這些 Meter 必須以互斥(mutually exclusive)的方式運作，即作用於不同的 Flow entry 的集合。此外，若需對同一組封包應用多個 Meter，則須通過多個連續的 Flow table 來實現。

Meter table 與 Group table 類似，它也是透過發送 Meter\_Mod 封包來組成一個或多個 Meter entry，其封包結構可參考圖 2-9，而 Meter entry 的結構可參考圖 2-10，其包含以下欄位：

1. Meter identifier：每個 Meter entry 會有其獨特的 ID，此 ID 不可重複。
2. Meter bands：為一個或多個 Meter band 所組成，Meter band 包含四個子欄位，包含 Band type、rate、counter、type，後面會進行更詳細的敘述。
3. Counters：記錄此 Meter entry 被執行幾次。

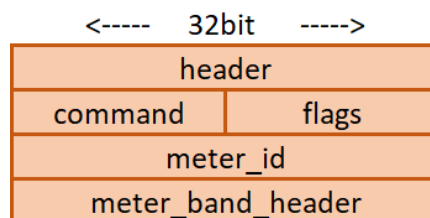


圖 2-9：Meter\_Mod 的封包結構

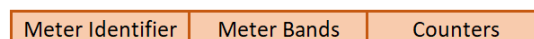


圖 2-10：Meter entry 之結構

Meter band 的說明如下：

- 1) Band type：用於定義 Band 的執行之行為，包含兩種選項：Drop 和 DSCP (differentiated services code point) remark，其中，Drop 表示直接丟棄封包，而 DSCP remark 用於設置封包丟棄的優先級，提供基本的 DiffServ policer 之功能。
- 2) Rate：即封包傳輸的最高限制速率，單位可以是 kbps (千位元每秒)或 Packet/s (每秒封包數)。值得注意的是，若是同一個 Meter entry 包含多個 Band，則會優先使用第一個 Band 所定義的速率限制。
- 3) Counter：記錄該 Band 被觸發執行的次數，以作為統計使用。
- 4) Type specific arguments：包含與該 Band 類型相關的其他參數，用於進一步補充設定。

本論文中使用 Band type 的 Drop 功能，以圖 2-11 為例進行說明。當封包進入交換機時，會先查詢 Flow table，尋找符合 Match field 條件的 Flow entry，當匹配到對應 Flow entry 後，可發現其 Action 指向 Meter table 中 Identifier 為 3 的 Meter entry，該 entry 定義的傳輸速率上限為 200 kbps，而如果流量的實際速率為 300 kbps 的，在經過 Meter table 的限制後，封包的最大傳輸速率將被限制為 200 kbps，這說明 Meter table 的機制能夠有效控制網路的傳輸速率，從而在網路壅塞發生時，來協助管理資料流的頻寬分配。

| Match                              | Priority   | Instruction              |
|------------------------------------|------------|--------------------------|
| eth_type=0x800,ipv4_dst="10.0.0.2" | Priority=1 | Actions=meter:5,output:2 |

| Meter Identifier | Meter Bands          | Counter |
|------------------|----------------------|---------|
| meter_id = 5     | type= DROP, rate=150 | 10      |

圖 2-11：Meter table 範例，上表為有 Flow entry 的 Flow table，下表為有 Meter entry 的 Meter table

## 2.3 常見的 SDN Controller 介紹

可支援 OpenFlow 協定的 SDN 控制器甚，可分為集中式(Centralized)與分散

式(Distributed)兩種類型[12]，集中式控制器常見的有 Beacon、NOX、OpenDaylight，而分散式的控制器則有 ONOS、HyperFlow、RYU 等，以下分別簡介這些控制器：

- 1) NOX 是首個開源的 SDN 控制器[4]，是由 Nicira 公司利用 C++語言所開發，該控制器基於 OpenFlow 1.0 的架構設計，早期獲得廣泛的社群支持，並引入了拓撲管理和交換機控制等多種概念，為後續控制器的發展奠定重要基礎；NOX 具備高性能，並在後期的版本 NOX-MT 支持多執行緒 (Multithreading) 操作，然而，由於其開發成本較高，加上隨著更簡化和高效的控制器逐漸出現，NOX 的使用逐漸減少，如今，其開源社群的討論活躍度已顯著降低。
- 2) Beacon 是一款廣受歡迎的 SDN 開源(Open source)控制器，由斯坦福大學與 Big Switch Network 於 2010 年開發[7]，其以 Java 語言所組成，Beacon 的主要目標是提升開發效率、實現高效能，以及支持應用程式的動態啟停，Beacon 採用多執行緒的架構，並為研究社群提供豐富的開發函式庫，其提供強大的記憶體管理與錯誤處理機制，主要接口包括 IRoutingEngine (支持最短路徑等路由模組)、ITopology (管理拓撲資訊)，以及 IBeaconProvider (與 OpenFlow 交換機互動)，此外，Beacon 通過 OSGi 支援執行期間模組化，使用戶能夠動態啟停應用程式或新增服務。
- 3) OpenDaylight 是由 Linux 基金會主導開發的一款開源 SDN 控制器，採用 Java 語言編寫，是當前最廣泛使用的控制器之一，其主要特點有模組化 (Modularity)、高擴展性(Scalability)、分散式架構(Distributed)，能夠適應各種網路環境需求，OpenDaylight 的開發得到眾多網路設備廠商的支持，因此在南向協議的支援上十分廣泛，包括 OpenFlow、OVSDB (Open vSwitch Database)、Netconf 等，並且能兼容多種網路設備。這些特性使其成為眾多廠商的首選 SDN 控制器。
- 4) Open Network Operating System (簡稱 ONOS) [8] 是由 Open Networking Lab 領導開發的一款分散式 SDN 控制器，與 OpenDaylight 一樣是透過社群驅動，



ONOS 的設計強調高一致性與卓越的性能，並提供狀態管理和控制器之間的任務協調功能，而與 OpenDaylight 聚焦於多功能與廣泛協議支援不同，ONOS 專注於運營商服務的開發，並致力於提供高度延展性，以適合服務供應商的大型網路環境。

- 5) Ryu [9] 是由日本 NTT 公司所設計開發的一款簡潔易用的 SDN 控制器，相較於其他控制器，Ryu 的架構更為直觀，但同時具備完善的 API，適合學術研究以及中小企業使用。Ryu 採用事件驅動程式設計(Event-driven programming)的架構，流程由事件推動運作，其運行方式如圖 2-12 所示，Ryu 的核心元件 Ryu-Manager 以非同步(Asynchronous)方式來運行，負責管理一個或多個應用程式。開發者的應用程式可透過事件處理器(Event handler)將事件放入事件佇列(Event queue)，佇列採用先進先出(First in first out)的方式，並由事件處理執行緒 (Event processing thread) 逐一處理事件，當某事件被觸發時，對應的事件處理器會進行相應的處理；此外，與資料路徑(Data path)相關的事件也會被註冊至事件佇列中來等待處理，其中的資料路徑指的是交換機連接至控制器後，於控制器內部所建立的記憶體結構，Ryu 的特性是高可用性(High availability)與低延遲(Low latency)。

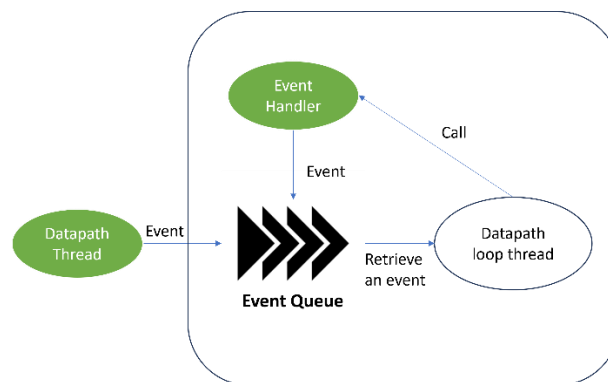


圖 2-12：RYU 運作架構

- 6) HyperFlow 是第一個為 OpenFlow 所設計的分散式控制平面(Control plane)，其設計靈感來自 NOX 控制器，HyperFlow 採用物理上分散、邏輯上集中化的

架構，透過多個地理上分散的控制器來實現高可用性，以解決集中式控制器因交換機數量增加所導致的流量壅塞問題；HyperFlow 使用發布/訂閱訊息系統(Publish and subscribe)來傳遞事件消息，並引入分散式檔案系統 WheelFS，以確保在網路分區時各控制器仍能正常運作；每個控制器僅能直接管理其連接的交換機，對於其他交換機的操作則需發布消息，包含來源控制器、目標交換機及本地指令等資訊，控制器之間也會定期傳遞訊息以維持網路狀態，倘若超過三個廣播間隔未收到訊息，該控制器會被認定故障，其管理的交換機將會遷移至其他控制器。

### 第三章 相關文獻探討

許多學者針對 SDN 管理的網路進行各種研究與應用探討，範疇涵蓋效率提升、負載平衡、擴展性、安全性、單一控制器管理、多控制器協作，以及不同架構對於網路各層面的影響等，文獻[14]致力於在提升互通性(Interoperability)的基礎上，改善多廠商環境中的 SDN 控制器；文獻[1]則回顧 SDN 解決方案及其於工業物聯網(Industrial Internet of Things)的應用，並針對多廠商架構在安全性、集中管理以及互通性方面進行系統化的分析，至於文獻[4]則提出一套以集中式方式來管理多網域資料中心網路，藉此改善並提升整體系統效益(Profit)之方法。

就 SDN 架構而言，可概略區分為集中式(Centralized)與分散式(Distributed)兩大類型[2]，特別來講，集中式指的是網域中所有網路設備僅交由單一個控制器來進行管理，而分散式指則可有多個控制器同時進行網域管理，文獻[2]亦指出集中式設計面臨可擴充性與單一弱點等問題，其可透過平坦式(Flat)或階層式(Hierarchical)的分散式架構加以改善；文獻[14]比較集中式與分散式 SDN 架構在物聯網環境下的應用效能，其結果顯示分散式架構可將負載分散至多個控制器，能夠大幅提升容錯能力(Fault Tolerance)與恢復效率；文獻[15]則採用隨機式(Random)、輪巡式(Round robin)與權重輪巡式(Weighted round robin)三種演算法，比較兩種 SDN 架構在反應時間(Response time)與傳輸效能(Transmission performance)上的差異，最終亦證實分散式架構在工作量分配後能取得更佳效能，有鑑於此，本研究將以分散式 SDN 架構作為主要方向，並探討控制器間的通訊機制(即東西向協議)，以確保全局網路狀態的一致性。

除了架構設計會影響 SDN 網路效能外，網路設備於在各鏈路(Link)間處理封包的機制同樣關係到使用者體驗，因此 QoS 亦成為重要研究課題之一，文獻[16]提出雙佇列系統(two queue system)，藉由回饋機制與多優先級佇列控制策略，不僅能改善資料與控制平面的通訊效能，同時也可提升 SDN 的延遲效能表現。

綜上所述，學者們的研究重點已逐漸聚焦於架構設計、負載平衡與 QoS 等關鍵面向，為更深入理解分散式 SDN 架構的潛在挑戰與解決方案，本章後續將依序探討分散式控制器間的通訊機制、控制器負載平衡所面臨的問題與對策、網路鏈路負載平衡的研究與實踐方法等議題，這些探討不僅能加深對分散式 SDN 架構的理解，也可作為跨網域協作與資源管理提供參考。

### 3.1 控制器間的通訊機制

相較於集中式架構僅透過單一控制器來管理整個網路狀態，分散式架構可同時使用多個控制器，而依其設計方式的不同，分散式架構可進一步細分為階層式 (Hierarchical) 與平面式 (Flat) 兩種類型[11]，其中，階層式通常將控制器劃分為 Master 與 Slave，Master 控制器負責協調所有 Slave 控制器，而 Slave 控制器則各自管理部分交換機；相較之下，平面式並無角色區分，所有控制器地位相同，當網路發生壅塞或其他異常情況時，各控制器會彼此協作以排解問題，圖 3-1 展示集中式的拓譜範例，而圖 3-2 與圖 3-3 則分別呈現平面式與階層式的分散式網路管理拓撲範例。

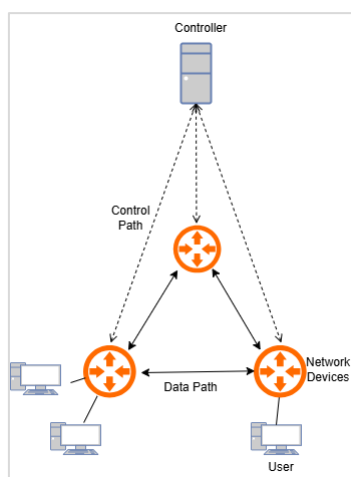


圖 3-1：集中式 SDN 網路管理拓譜的範例

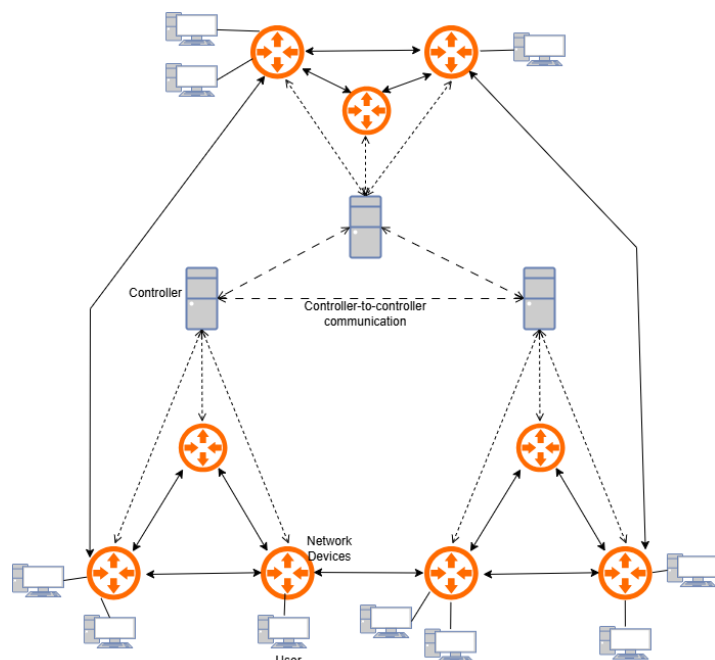


圖 3-2：平面式的分散式 SDN 網路管理拓譜範例

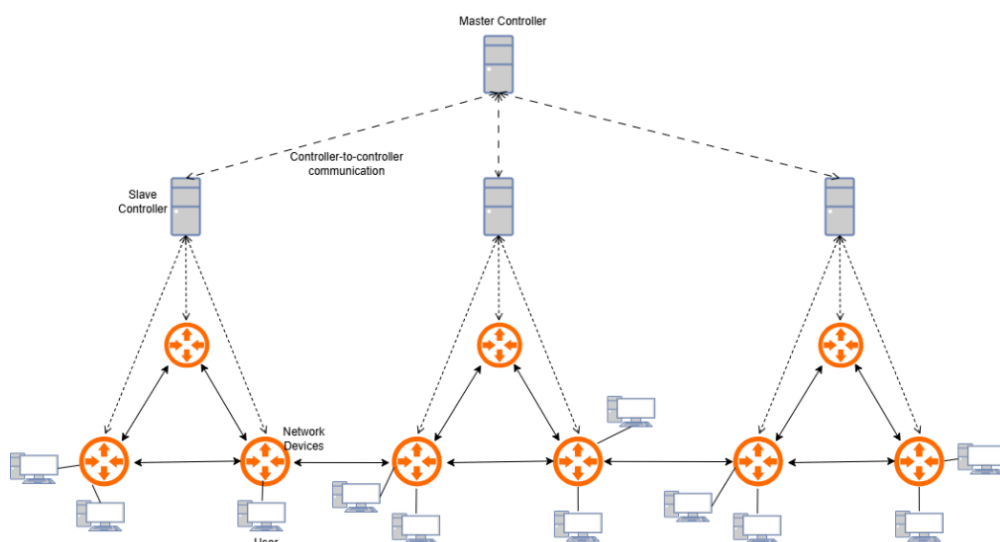


圖 3-3：階層式的分散式 SDN 網路管理拓譜範例

分散式 SDN 架構的優勢之一在於能將整體網路切分為多個網域，並交由各網域的 SDN 控制器進行可程式化管理，為使多個控制器能夠相互進行協作，需要透過東西向通訊(East-west communication)來共享網路狀態亦即傳遞控制資訊，為此，已有不同的通訊協定或機制被提出，舉例來講，HyperFlow 採用發布/訂閱(Publish/Subscribe)模式，允許控制器之間共享網路狀態資訊，進而促成分散式控制平面的協同運作 [10]，Onix 平台則藉由分散式多重資料庫(Replicated database)

與分散式雜湊表(Distributed hash table)的設計，使多個控制器得以在大型網路環境中共享狀態資訊，以提升可擴展性與容錯能力 [18]；此外，OpenDaylight 與 ONOS 控制器都使用 Raft 共識演算法(Raft consensus algorithm)以進行資料共享 [19]，然而，這些方法在高負載環境下仍會面臨部分可擴展性與安全性問題，為應對此挑戰，Benamrane 等人提出一個通訊介面，以支援多個分散式控制器的同步、通知交換以及服務共享 [20]，實驗結果顯示他們的通訊界面能夠實作分散式防火牆與負載均衡等服務，並在實際廣域網路拓撲中的效能優於基於集群(Cluster-based)的模型。

### 3.2 控制器負載平衡

在分散式 SDN 網路當中，控制器的負載平衡是一項重要的研究議題，依其設計的差異，大致可分為階層式負載平衡與平坦式負載平衡兩大方向，階層式架構下，控制器之間無法直接與同層級成員進行溝通，而是需要透過上層或主控制器(Root/Master controller)來進行協調 [24]，主控制器負責監管所有下層控制器，而下層控制器則會將底層資訊抽象化後傳遞，以避免向上傳遞過多未經處理的資料，從而減少主控制器的過載風險 [21]，於此基礎上，文獻[21]更進一步將控制平面細分為「負載平衡控制層」(Load balancing control plane)與「分散式控制層(Distributed control plane)」，透過分析網路狀況並權衡優先級，來為各控制器分配合適的流量，以達到有效的負載平衡；此外，文獻[22]提出基於動態閾值調整的控制器負載平衡演算，可依據當前網路狀況來動態調整控制器的負載閾值，並在負載較高的控制器與負載較低的控制器之間進行交換機遷移，以因應流量不均衡的情形發生，上述方法皆藉由多個控制器間的合作，來避免單一控制器形成系統瓶頸或故障點；文獻[23]進一步在分散式 SDN 架構中採用階層設計，將控制器分為 Master 與 Slave 的層級，並由 Master 控制器負責系統的負載平衡與容錯機制，而當部分控制器發生故障時，系統能迅速切換，而若是 Master 發生故障時，則由

Slave 控制器中選出一個候選者升級為新的 Master，以確保整體網路仍可穩定運行。

除了階層式的負載平衡，平坦式架構同樣受到學者關注，文獻[25]提出多閾值負載平衡的交換機遷移方案，透過將控制器負載區分成多個等級，當某控制器的負載明顯高於(或低於)其他控制器時，便會觸發交換機遷移機制，並動態調整閾值，此方式能夠減少不必要的更新操作並降低控制器間的通訊開銷，實驗結果顯示其在響應時間(Response time)、開銷及吞吐率等指標上，該研究所提出的方式皆展現出良好的整體效能。

整體而言，分散式控制器負載平衡的研究重點，主要圍繞在如何透過控制器的互相協作，來降低單點故障風險並提升網路效能，而無論是階層式或是平坦式架構，都嘗試以動態調整、交換機遷移或層級劃分等方式，以確保控制器在高流量或故障狀態下依舊能穩定運行，然而，上述文獻多半著墨於控制器層面的負載平衡，尚未深入探討資料平面(Data plane)之間的流量分配與路徑規劃，有鑑於此，下一節將轉而聚焦資料層的負載平衡機制，進一步探討如何在分散式 SDN 架構下針對鏈路流量進行優化。

### 3.3 網路鏈路負載平衡

鏈路負載平衡技術的核心在於如何將網路流量平均分配至多條鏈路，避免因許多流量集中在少數鏈結而引發鏈路中壅塞(Congestion)，以代提升網路的吞吐量與穩定性 [24]，部分研究透過監測頻寬利用率(Bandwidth utilization rate)、封包遺失率(Packet loss rate)、信任值以及傳輸跳數(Hop count)等指標，來識別低負載路徑以分攤流量，從而減少某些鏈路的過度擁塞 [25]。

另有研究於無線網狀網路(Wireless mesh network)環境中，提出一套考量服務品質(QoS)限制的負載平衡路由演算法 [26]，該方法於集中式 SDN 控制架構下整合了訊噪比(SNR)、封包傳送率與功率參數等資訊，以進一步增進網路性能；文

獻[28]則提出利用 SDN 來提升影片播放品質，其藉由流量工程(Traffic engineering)機制並結合 QoS 保障的概念，將流量區分為「延遲敏感」與「非延遲敏感」兩類，高優先權的延遲敏感流量會被導向可確保頻寬的鏈路，而非延遲敏感流量則分配至其他鏈路，以優化整體效能並提升使用者體驗，然而，這些研究大多聚焦於集中式 SDN 架構，尚未納入多網域環境的複雜性。

亦有學者針對多網域 SDN 網路的路由進行相關研究，例如文獻[40]提到將東西向界面用於跨網域資料流的控制，允許 SDN 控制器覆寫 BGP 行為並探索多條可用的跨網域路徑；文獻[41]提出 TRAQR 方法，結合區塊鏈技術與 SDN，在多域環境中提供信任感知的端到端 QoS 路由，這種設計透過區塊鏈確保跨域資訊透明與不可竄改，強化路由決策的可靠性與安全性，並可兼顧 QoS 要求，然而，上述方法並未考量到起始主機與目標主機處於同網域的情況下，能否進行跨網域流量協同管理以來實現負載平衡。

文獻[3]著眼於多網域 SDN 網路的管理方式，提出跨網域協作傳輸(Collaborative route management)的概念。當某一網域的負載變得過高時，可將部分資料流轉移至閘道器(Gateway switch)，再利用其他網域的轉送機制，將封包傳遞至原始網域的另一個閘道器，以繞開壅塞鏈路，並緩解當前網域的過載情況，然而，該研究並未探討當協助轉送的網域本身也面臨鏈路阻塞時，應如何有效處理受助的資料流，而後續研究[4]則引入異質資料流之概念，其透過權重分配以確保自家網域之資料流的穩定傳輸，同時為受助資料流提供最低頻寬保障，以避免發生資源飢餓(Starvation)並提升整體網路效能，但文獻[3]與 [4]並未明確指出資料流送出原網域後原網域發生壅塞時該如何處理，有鑑於此，本研究將完善此部分並新增價值之概念，並在後面進行探討。

### 3.4 討論

綜合上述文獻可知，SDN 在網路管理上具備多項優勢，包含透過控制器取得全域資訊、能動態更新網路規則、以北向介面開發多樣化應用，以及經由南向介



面與網路設備進行互動，相比集中式 SDN，分散式架構提供了更高的擴充性，並有效避免單一故障點的風險；而就網路流量控制而言，若能及時將流量導向較低負載的鏈路，便可減輕特定鏈路的擁塞，同時，多網域間的協作亦能透過借用其他網域的路徑，降低自身網域的壅塞程度，除了基本的流量疏導，網路管理者也可考量資料流的價值，適度為高價值的資料流分配更多頻寬，不過，若單純只考量價值則會容易造成低價值流量餓死，兩者會相互衝突，如何在多元且動態的網路環境中權衡兩者的重要性，要注重價值的分配方式或是要平均分配鏈路頻寬，或是要進行其他的選擇，仍是一個須深入探討的議題。

## 第四章 問題定義

本研究考量如校園或公司等大型網路環境，為方便管理這類型的大型網路，通常會將網路切分成多個子域(Domain)，隨著網路規模變大，傳統網路架構無法妥善使用網路資源，這時可以透過 SDN 來協助進行進行管理，其中，每個網域擁有獨立的控制器，負責管理其所屬網域的網路資源，並可透過多台網關交換機 (Gateway switch)與其他網域進行連接而溝通，使得跨網域通訊(Cross-domain communication)得以實現。

本研究的重點在於跨網域的流量管理，藉由 SDN 控制器的集中式管理能力，在多網域環境下動態調整流量傳輸策略，使得各網域能夠協同合作，以達到最大的運輸效益，其中一個重要的特性就是使資料流可以向別的網域借路，再回到其原始網域。

值得注意的是，在過去 SDN 架構中，控制器僅能根據自身網域的網路狀態來進行路由決策，當某個區域發生網路壅塞時，流量僅能在本網域內尋找可行路徑，此時若是找不到任何替代路徑時，就只能放任讓鏈路壅塞，進而拉高封包遺失率，至於在具可協同合作的多網域環境下，當某個網域內發生壅塞時，可以考慮經由其他網域繞道傳輸，確保流量能夠順利抵達目的地。

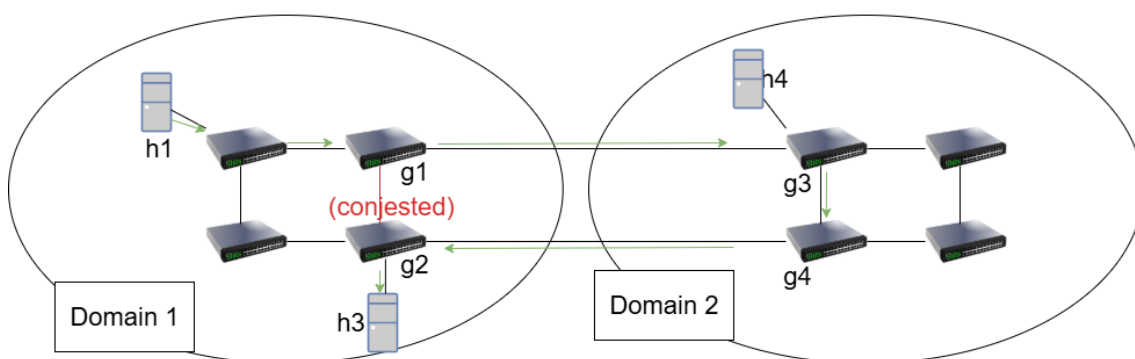


圖 4-1：在具協同合作的多網域下的封包繞率之範例

我們以圖 4-1 來舉例說明，當網域 1 發生壅塞，且 h1 需要將流量傳輸給 h3 時，傳統方法由於侷限只能在網域內尋找路徑，此時會造成大量封包遺失，事實

上，網域 1 的控制器可考慮向網域 2 借路，特別來講，流量的傳輸順序為 h1 先將流量發送至 g1 (網域 1 的 Gateway switch 1)，再由 g1 轉發至 g3 (網域 2 的 Gateway switch 3)，並進一步經過 g4 (網域 2 的 Gateway switch 4)，最後返回網域 1 內部，經由 g2 (網域 1 的 Gateway switch 2)將流量傳送至 h3，因此路徑將為 h1->g1->g3->g4->g2->h3，在此範例中，網域 1 為原始網域，網域 2 為協助網域、對於網域 2 而言，該資料流稱之「協助流」(Assistance flow)。

上述的策略有效降低網域 1 內部的壅塞情況，並提升整體網路效能，然而，網域封包傳輸過程會有不穩定性，且傳輸的路徑會拉長，因此需在必要時再啟動借路機制，以減少過度的路由與相關成本，另外，跨網域流量繞道會涉及到網域間的頻寬資源分配，為避免某個網域的流量過度消耗其他網域的資源，我們還需要讓協助網域能判斷是否要進行協助以及要提供多少協助。

除此之外，我們也考慮資料流具有不同的價值(Profit)，在某些應用中，流量會有重要性之分，比較重要的流量通常也會有較高的經濟價值，較高的 Profit 值意味著應需給予更高的頻寬保障，以提供更高的商業貢獻，所以當無法透過跨網域流量繞道來減輕壅塞時，網域內需要有控制的進行個別資料流的限速，以確保所獲得的 Profit；本研究的主要目標可透過以下公式來表示：

$$\max \sum_{\forall f_i \in \hat{F}}^N P_i \times \left( \frac{b_i^a}{b_i^d} \right) \quad (1)$$

此公式(1)中，每條資料流定義為  $f_i$ ， $\hat{F}$  為全部資料流的集合， $b_i^d$  為資料流  $f_i$  所要求的頻寬、 $b_i^a$  為  $f_i$  資料流被分配到頻寬、 $P_i$  為  $f_i$  資料流的 Profit 值，也就是沒有丟失封包情況下最高可獲得的價值。

然而，若單純依據價值進行頻寬分配，雖然高 Profit 值流量將獲得較多的資源，但低 Profit 值流量則可能被忽略，甚至無法獲得頻寬，導致流量發生 Starvation。為解決此問題，我們也應確保所有流量至少能獲得一定的頻寬，以避免過度傾斜的資源配置，因此本系統限制條件：

$$0 \leq 1 - \frac{b_i^a}{b_i^d} < 1, 0 \leq b_i^a \leq b_i^d, \forall f_i \in \hat{F} \quad (2)$$

在公式(2)中，我們要確保沒有資料流的封包丟失率會到達 100%，以及任何資料流被分配的頻寬不會超過它所要求的頻寬，這表示若有資料流的頻寬要求被滿足，多餘的頻寬可以用來滿足其他價值較低的資料流，以此提升頻寬利用率，舉例來說，若有兩個資料流在一個最大頻寬 8 Mbps 的鏈路上形成壅塞，資料流 A 要求 5 Mbps 的頻寬，提供 120 Profit 值，而資料流 B 要求 5 Mbps 的頻寬，提供 20 Profit 值，若直接使用 Profit 值來進行等比例的分配，則資料流 A 分配到  $\frac{120}{120+20} \times 8 = 6.857$  Mbps，資料流 B 分配到  $\frac{20}{120+20} \times 8 = 1.143$  Mbps，可是資料流 A 只需要 5 Mbps，系統要能夠將多餘的頻寬分配給其他資料流以善用頻寬資源並提升整體 Profit 值的獲取。

此外，為使協助網域在壅塞時還願意提供頻寬給協助流，應設計一個機制使協助流原網域分享部分 Profit，激勵網域間的互相協助。

總結以上說明，本論文共有以下幾點目標：

- 1) 當鏈路發生壅塞時，可以透過控制器檢查出受到影響的資料流。
- 2) 對於受到影響的資料流，可以重新分配路徑或進行跨網域借路來舒緩壅塞。
- 3) 當鏈路還是壅塞且無法進行跨網域借路時，進行流量速率的調節，以確保整體系統 Profit 值得獲取。
- 4) 提升系統整體的所獲得的 Profit 值，同時避免低 Profit 值的資料流餓死。

以下列出本研究中使用的參數表：

| 參數        | 解釋                                        |
|-----------|-------------------------------------------|
| $\hat{F}$ | 資料流集合( $\forall f_i \in \hat{F}$ 表示某條資料流) |
| N         | 總 Profit 數量                               |
| $P_i$     | 資料流 $f_i$ 的 Profit                        |
| $b_i^d$   | 資料流 $f_i$ 需要的頻寬                           |
| $b_i^a$   | 資料流 $f_i$ 被分配的頻寬                          |
| $\sigma$  | 系統中表示壅塞的門檻值                               |

表 4-1：本研究使用的參數

## 第五章 研究方法

本論文利用 SDN 技術，針對最大化價值並多網域協作的環境中提出名為 Profit negotiation collaborative flow management (PNCFM)的方法。當自身網域的流量發生網域壅塞時，其控制器能判斷是否要在自身網域解決或抑是向其他網域請求協助來提升整體網路的效能，因此有分為自身網域的资料流與協助其他網域資料流的協助流，當只處理自身網域的流量時使用 Profit maximization 模組，而須處理自身資料流與協助流時則採用 Nash Bargaining 模組。

本研究具體方法簡述如下：

- (1) 控制器定期詢問其轄下的 Switch 資訊，計算資料流的封包遺失率。
- (2) 當資料流的封包遺失率超過預設的門檻值時，控制器將進行鏈路的負載平衡，設法改變路徑，將資料流轉移到同網域中負載較輕的鏈路。
- (3) 由於網域的頻寬資源有限，若在自身網域中無法透過改變路徑來改善封包遺失率，控制器將對外尋求協助傳輸，藉此改善網路壅塞的情況。
- (4) 控制器的工作是確保其網域中的自身資料流的傳輸品質與價值產出，由於協助其他網域的緣故，每個網域中可能有來自其他網域的協助流，若發生壅塞時必須確保自身資料流的傳輸。這部分會透過基於 Nash Bargaining Solution 的方法來達成，將於後面章節進行更詳細的說明。
- (5) 資料流經濟價值(Profit)的高低之分，因此須改用 Profit 來做為流量分配的指標，高 Profit 的資料流有較多的頻寬。
- (6) 除考量經濟價值外，也需確保資料流不會發生 Starvation，因此引入懲罰機制，系統會避免某些資料流拿不到頻寬。

圖 5-1 顯示本研究所設計之 PNCFM 方法。首先，系統會初始化網路拓撲並完成各網域控制器之間的通訊設定，相關細節詳見 5.1 節，完成初始設定後，各控

制器將定期統計所屬網域內各交換機的即時流量資訊，並藉由 5.2 節所提出的監控機制來識別壅塞現象。

當偵測到某網域內發生壅塞時，控制器會根據第 5.3 節所描述的機制，進一步判斷該網域是否要向其他網域請求協助、以及其他網域該如何判斷是否給予協助。

當網域內沒有來自其他網域的資料流時 (也就是協助流)，其控制器會透過 5.5 節的 Profit Maximization 模組，依照該鏈路最大頻寬與資料流資訊進行價值最大化且避免資料流餓死的頻寬分配。

另一方面，當網域內存在協助流時，該網域的控制器會透過 5.4 節所描述的 Nash Bargaining 模組來判斷是否要繼續提供協助並且要提供多少協助，該網域會基於已知資訊判斷利潤分配固定的情況下該如何最大化利潤，而在對協助流限速後，網域內還會有其他資料流，因此在扣除分配給協助流的頻寬後，會再執行 5.5 節的 Profit Maximization 模組。

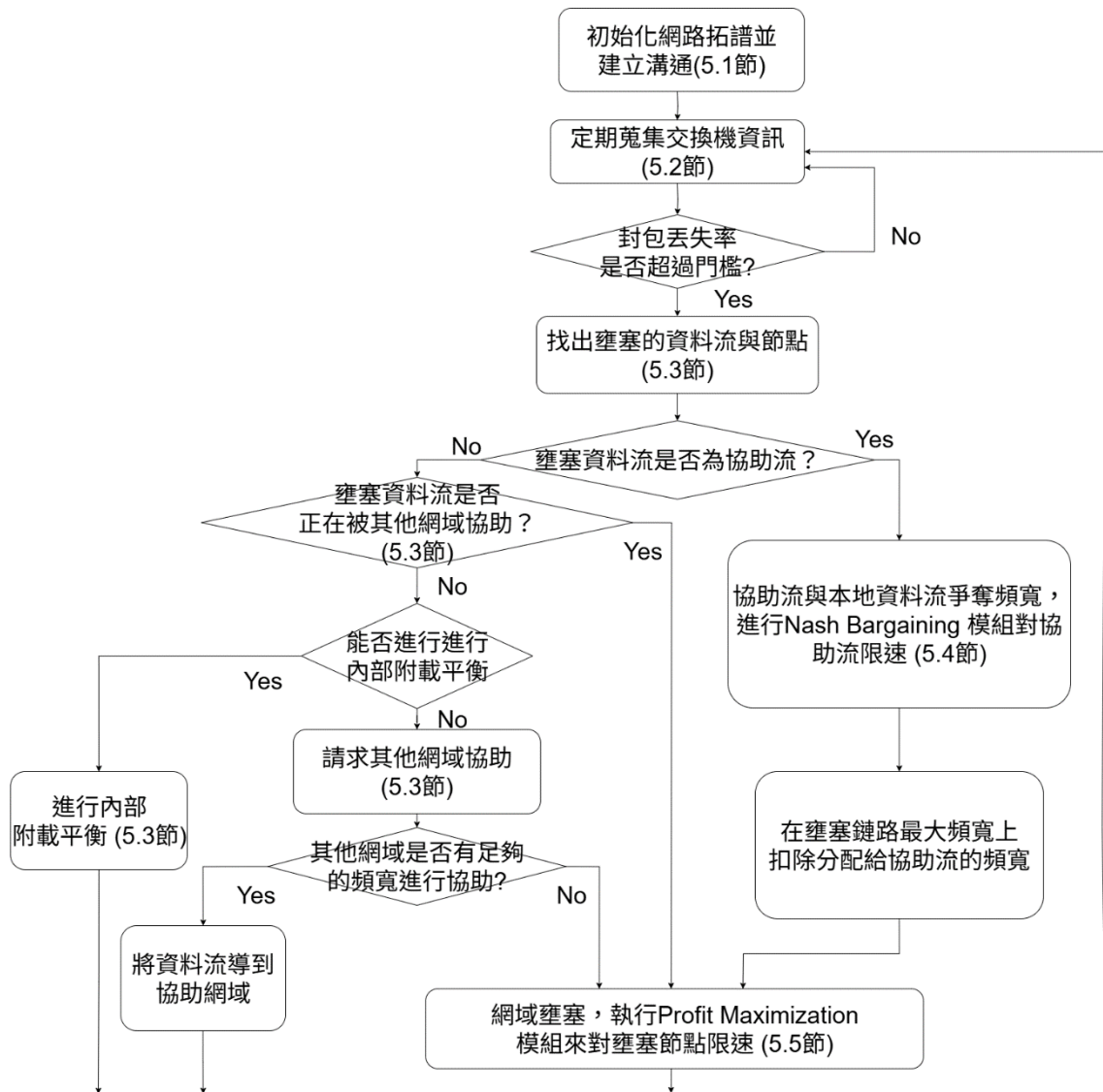


圖 5-1：本系統之流程圖

## 5.1 網路拓模初始化與跨網域溝通

本論文考量具有多個網域的 SDN 網路，以模擬大型公司或校園網路，每個網域由一個控制器進行管理。這樣的規劃是因為公司或學校的內部網路中，各單位會希望管理自己的子網路，故會根據每個部門的情況切分成不重疊的子網路；我們以圖 5-2 為例，控制器 1 負責網域 1 且管理交換機 s1、s2、s3、s4，控制器 2 負責網域 2 並管理交換機 s5、s6、s7、s8，至於控制器 3 負責網域 3 且管理交換機 s9、s10、s11、s12，此外，Gateway switch 則是連接兩個網域的交換機，交換



機之間亦會透過傳送 Link Layer Discovery Protocol (LLDP) 封包來得知鄰居的資訊，其中，s1 可以視為網域 1 對網域 2 的 Gateway switch，s7 可以視為網域 2 對網域 1 的 Gateway switch。

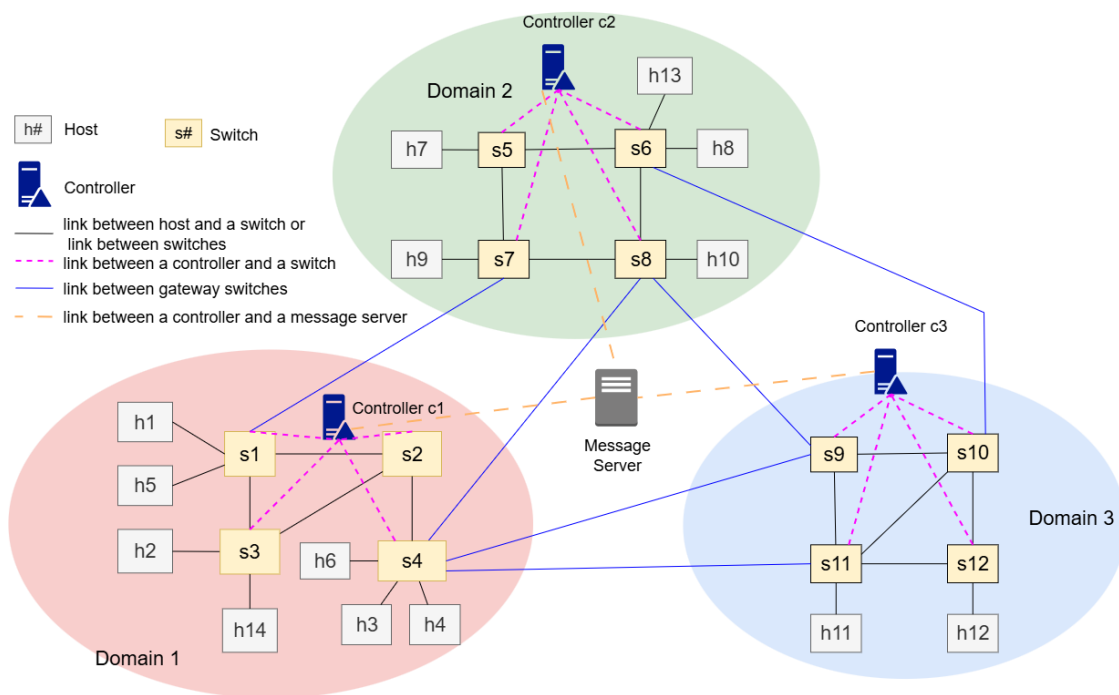


圖 5-2：多網域 SDN 網路的拓譜範例

控制器之間的溝通則是透過 ZeroMQ 的套件來進行，其會建立 Subscriber/Publisher 的架構[3]，所有控制器會訂閱到一個 Message server，定期去該 Server 上查找有無新資訊，當收到新訊息時，控制器就會檢查並進行對應的動作；本論文中透過三種訊息來進行跨網域合作，訊息的相關參數可以參考表 5-1，以下對它們進行詳細說明：

1. Ask\_Help：當網域發生壅塞且網域內沒有頻寬足夠的替代路徑時，控制器會發送此訊息以請求其他網域的協助。該訊息內包含資料流離開網域的 gateway switch “Out\_Sw”、回歸網域的 Gateway switch “In\_Sw”、需求頻寬“BW”、價值 “Profit”、願意在壅塞時提供的“Slice”值。

2. Can\_Help：當控制器收到 Help 訊息並判斷這是自己網域內可以處理的資料流後，就會產生此訊息並發布到 Message server 上給請求協助的網域，訊息中包含能幫忙轉送的交換器，其中，“In\_Sw”為對於協助網域而言協助流要進來的 Gateway switch，“Out\_Sw”則為對於協助網域而言協助流離開網域的 Gateway switch。
3. Rate\_Limit\_Notify: 由網域內有協助流的控制器所傳送，由於網域內發生壅塞，無法完整借路給別人，所以要對協助流進行限速，此訊息內包含該網域願意借出的頻寬。
4. Profit\_Registration: 由管理方網域之控制器所傳送，藉此動態修改資料流的優先權。

| Message             | Parameter                                                                                                                                                                                                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ask_Help            | <ul style="list-style-type: none"> <li>• Domain: 發訊息之網域</li> <li>• In_Sw: 資料流回歸網域的 Gateway switch</li> <li>• Out_Sw: 資料流離開網域的 gateway switch</li> <li>• Src_IP: 資料流的起始主機 IP</li> <li>• Dst_IP: 資料流的目標主機 IP</li> <li>• BW: 資料流占用頻寬</li> <li>• Profit: 資料流價值</li> <li>• Slice: 協助網域壅塞時，願意分享的 Profit 之比例</li> </ul> |
| Can_Help            | <ul style="list-style-type: none"> <li>• Domain: 此訊息的目標網域</li> <li>• In_Sw: 協助網進來協助網域的 Gateway Switch</li> <li>• Out_Sw: 協助流離開協助網域的 Gateway Switch</li> <li>• Src_IP: 協助流的起始主機 IP</li> <li>• Dst_IP: 協助流的目標主機 IP</li> </ul>                                                                                      |
| Rate_Limit_Notify   | <ul style="list-style-type: none"> <li>• Domain: 此訊息的目標網域</li> <li>• Src_IP: 協助流的起始主機 IP</li> <li>• Dst_IP: 協助流的目標主機 IP</li> <li>• Help_BW: 協助網域願意支援該資料流的頻寬</li> </ul>                                                                                                                                           |
| Profit_Registration | <ul style="list-style-type: none"> <li>• Domain: 此訊息的目標網域</li> <li>• Src_IP: 資料流的起始主機 IP</li> <li>• Dst_IP: 資料流的目標主機 IP</li> <li>• Profit: 資料流的 Profit 值</li> </ul>                                                                                                                                              |

表 5-1：控制器訊息之參數表

## 5.2 流量定期監控

當交換機與控制器建立連線後，控制器便可透過 EventOFPSwitchChange 函數來掌握交換機的狀態，該狀態共可分為四種類型，詳列於表 5-2，特別來講，MAIN\_DISPATCHER 表示交換機維持穩定連線，至於 DEAD\_DISPATCHER 則代表連線已中斷，另一方面，控制器也可定期透過 OFPFlowStatsRequest 及 OFPPortStatsRequest 指令來向交換機查詢各項統計資料，這包括 Flow table 中每條 Flow rule 的封包處理統計，以及各個 Port 的容量、收到與送出傳輸量[30]，透過這些資訊，控制器能推算出各鏈路的剩餘頻寬，進而建立可用的路徑資訊以供後續決策模組使用。

| 狀態                   | 意義                                              |
|----------------------|-------------------------------------------------|
| HANDSHAKE_DISPATCHER | 傳送並等待 Hello 訊息                                  |
| CONFIG_DISPATCHER    | 協商版本後傳送 features-request 訊息                     |
| MAIN_DISPATCHER      | 接收到 switch-features 訊息後，傳送 set-config 訊息，也表示連線中 |
| DEAD_DISPATCHER      | 中斷斷線，可能因無法復原的錯誤而發生                              |

表 5-2：交換機之狀態表

本研究設計四種資料表，用以儲存與各網域相關的資訊，分別為主機交換機位置表 (Host switch location table)、資料流路徑紀錄表 (Flow path table)、資料流價值表 (Flow profit table)、資料流交換機情報表 (Flow switch information table)，分別說明如下：

1. 主機交換機位置表：該表用以記錄網域中所有主機的相關資訊，包括每台主機的 IP 位址、MAC 位址，以及其所連接的交換機與 Port，表 5-3 給予了一個範例，值得一提的是，若是某主機曾與其他網域的主機建立過連線，其對應的外部主機資訊也會同步記錄於此表中。

而由於本地主機無法直接對應到非本網域的交換機上的實體 Port，系統會以 Gateway switch 作為代理來進行記錄；當主機尚未與控制器建立完整對應關係時，系統會對尚未註冊之交換機 Port 及 Gateway switch 發送 ARP (Address resolution protocol) 請求，以查詢指定 IP 位址所對應的 MAC 位址，藉由觀察 Gateway switch 是否將 ARP 請求轉送至外部網域，即可推測通往外部主機的路徑所經之交換機與 Port，舉例而言，如表 5-3 所示，IP 為 10.0.0.1 的主機位於外部網域中，控制器觀察到其需透過 Gateway switch s7 的 Port 3 才能建立連線，因此在位置表中將其對應關係記錄為 (s7, 3)。

| (IP address, MAC address)          | (Switch, Port) |
|------------------------------------|----------------|
| ('10.0.0.7', '00:00:00:00:00:05')  | (s5,4)         |
| ('10.0.0.8', '00:00:00:00:00:06')  | (s6,4)         |
| ('10.0.0.9', '00:00:00:00:00:09')  | (s7,4)         |
| ('10.0.0.10', '00:00:00:00:00:0A') | (s8,6)         |
| ('10.0.0.1', '00:00:00:00:00:01')  | (s7,3)         |
| ('10.0.0.7', '00:00:00:00:00:07')  | (s8,4)         |

表 5-3：主機交換機位置表範例 (以 Domain 2 為例)

2. 資料流路徑紀錄表：為了建構各網域中交換機之間的路由資訊，我們利用 NetworkX 函式庫 [33] 來建立網路拓樸，並根據此拓樸推導出各條資料流的最短路徑 (Shortest Path)，即跳數 (hop count) 最少的路由，表 5-4 中列舉了三個範例，其中，每個控制器都有自己的一個表，控制器會根據資料流的來源與目的主機所在位置來生成其最佳傳輸路徑。

| Controller1 |         | Controller2 |         | Controller3 |         |
|-------------|---------|-------------|---------|-------------|---------|
| Flow        | Path    | Flow        | Path    | Flow        | Path    |
| h13→h10     | [3,1]   | h13→h10     | [7,8]   | h8→h11      | [10,11] |
| h1→h4       | [1,2,4] | h9→h8       | [7,5,6] |             |         |
|             |         | h7→h8       | [5,6]   |             |         |

表 5-4：資料流路徑紀錄表範例(三個網域各自的表)

我們以第一筆資料流為例，若是來源與目的主機皆位於同一網域，如圖 5-3 中網域 1 的主機 h1 至 h4 (以 h1→h4 表示)，則其路徑僅會經過該網域內部的交換機，透過最短路徑計算，該資料流會依序經過 s1、s2、s4，對應路徑表內為 [1,2,4]。

相反地，若資料流需跨越不同網域 (即來源與目的主機不屬於同一個網域中)，則會分別建立來源網域與目的網域的獨立子路徑，此過程分為兩段，其中，來源網域內的路徑從連接來源主機的交換機起，經過中繼節點，最終抵達本網域的 Gateway switch，至於目的網域的子路徑則從該網域的 gateway Switch 開始，傳送至目的主機所連接的交換機為止，用圖 5-2 的拓譜舉例，以一個從 h13 到 h10 的資料流 (用 h13→h10 表示)來說，來源主機 h13 位於網域 1，其內部路徑為 s3→s1，而目的主機 h10 所在的網域 2 則對應路徑 s7→s8，因此整體路徑將會是 h13→s3→s1→s7→s8→h10，反映資料流跨越多個網域的傳輸歷程。

3. 資料流價值表：此表為了反映不同資料流對系統所產出的經濟價值，特別來講，價值的數值範圍為 $p_{min}$ 至 $p_{max}$ ，其中，數值越高則代表該資料流具有較高的價值產出，且透過本論文的頻寬分配方法，它們也可以在鏈路壅塞時有機會獲得較高的頻寬資源佔比，至於資料流的價值則由網路管理員所預先設定，並可在需要時進行實時動態修改。

而對於跨網域的資料流，其價值設定由來源主機所屬網域的控制器負責指定，網路管理者可透過一則名為 Profit\_Registration 的訊息主動將優先

權設定資訊傳送給該網域的控制器，內容包含來源與目的主機 IP 位址，以及對應的 Profit 值。舉例來說，若一個從 h1 到 h4 的資料流被指定 Profit 為 80，則管理者將會向控制器 1 傳送以下訊息：Message = “Profit\_Registration”, Domain = 1, Src\_IP = “10.0.0.1”, Dst\_IP = “10.0.0.4”, Profit= 80。

當發生跨域協作，即某網域需借用其他網域路徑來進行資料流傳輸(即協助流)，該來源網域的控制器亦會透過 Ask\_Help 訊息中的 Profit 欄位，來通知協助方的控制器該筆資料流的價值，以協助其進行路由選擇與資源配置評估，相關應用情境與決策機制將於 5.4 節中進一步說明。

4. 資料流交換機情報表: 上述三個表可幫助控制器了解資料流的相關資訊，但是不包含實際傳送封包時的傳送狀況，因此每個網域的控制器會定期向其轄下的交換機發送 OFPFlowStatsRequest 訊息，以掌握各條資料流的封包傳輸情況並更新於此表內，該表依序會存 Src\_Ip、Dest\_Ip、switch\_id、Out\_Port、Send\_Packet 等欄位；其中，Src\_Ip 為發送的主機，Dest\_Ip 為目的地的主機，switch\_id 為當前這筆資料代表的交換機，out-port 為當前這筆資料中所代表的交換機以此 Port 送出這條資料流的封包，Send\_Packet 代表從 Src\_Ip 到 Dest\_Ip 的這條資料流從對應的交換機與輸出埠上送出多少封包，詳細範例可參考表 5-5。

| Src_IP   | Dst_IP    | Switch_Id | Out_Port | Send_Packet |
|----------|-----------|-----------|----------|-------------|
| 10.0.0.7 | 10.0.0.8  | 5         | 1        | 3241        |
| 10.0.0.7 | 10.0.0.8  | 6         | 1        | 3230        |
| 10.0.0.7 | 10.0.0.10 | 5         | 2        | 3265        |
| 10.0.0.7 | 10.0.0.10 | 7         | 2        | 3241        |
| 10.0.0.7 | 10.0.0.10 | 8         | 6        | 2607        |

表 5-5：網域 2 資料流交換機情報表範例

透過分析資料流從來源主機所連接的交換機( $s_k$ )到目的主機所連接的交換機( $s_j$ )之間的封包傳輸差異，即可推估該資料流的封包遺失情形，用來記錄每條資料流在每台交換機封包送出的狀況。

具體而言，有一個資料流  $f_i$ ，若以  $E_k$  表示  $s_k$  在特定查詢時間段  $t$  針對資料流  $f_i$  所累計的封包發送數量， $E_j$  表示  $s_j$  在同一時間段所對應同資料流傳送出去的封包數量，則資料流  $f_i$  的在該時間段的封包遺失率 (Packet loss rate) 可藉由下列公式進行估算：

$$R_{i,t} = \frac{E_k - E_j}{E_k} \times 100\% \quad (5.1)$$

此外，若資料流的來源或目的主機並不位於同一網域內，舉例來講：在協助流的傳輸過程中，則系統將以該主機所對應的 Gateway switch 作為其在網域中的代表節點來進行後續處理，對於資料流交換機情報表整理後可得到類似於表 5-6 的內容以方便在本系統中計算封包丟失率。

| (Src_IP ,<br>Dst_IP) | S5   | S6   | S7   | S8   | Loss rate |
|----------------------|------|------|------|------|-----------|
| h7→h10               | 3265 |      | 3241 | 2607 | 20%       |
| h7→h8                | 2343 | 2301 |      |      | 1.7%      |
| h2→h4                |      |      | 3880 | 1040 | 73%       |

表 5-6：網域 2 的流量紀錄與封包丟失率之計算範例

以表 5-5 所列資訊為例，針對資料流 h7→h10，由於兩台主機分別連接至本網域內的 Switch S5 與 S8，其封包遺失率可透過兩端累積封包數量差進行計算，對應公式為： $(3265-2607) / 3265 \times 100\%$ ，計算結果約為 20%，同樣地，資料流 h7→h8 的遺失率可依據 Switch S5 和 S6 所統計之封包數得出，其結果為： $(2343 - 2301) / 2343 \times 100\%$ ，約為 1.7%。而至於 h2→h4 這筆資料流，屬於來自網域 1 的協助流，因此在本網域內的對應節點為 Gateway switch S7 (入口)與 S8 (出口)，根據其封包數據，遺失率為： $(3880-1040) / 3880 \times 100\%$ ，約為 73%。

## 5.3 壅塞與跨域負載平衡

在本研究中，控制器會判斷壅塞發生時有兩個條件：(1)有資料流在某一時間段的封包丟失率超過門檻值 $\sigma$ ，(2)在該資料流會經過的路徑中，發生壅塞的鏈路存在兩條(或以上)的資料流，此時，控制器會選擇其中一條資料流當作要處理的對象(稱為問題流)，具體來講，控制器發現有資料流的封包丟失率超過 $\sigma$ ，首先會比對流交換機情報表(參考表 5-5)，以找出在同一鏈路的其他資料流，在此系統中 — 在排除來源主機與目的主機不同網域的情況後 — 資料流會有三種情況：(1)正常傳送的資料流、(2)跨網域到其他網域借路的資料流、(3)其他網域來當前網域借路的協助流，而在壅塞鏈路上，控制器會依照(3)>(2)>(1)的情況來選擇資料流當成問題流，因為他們是當前造成壅塞時最應該先處理或是他們就是問題來源的資料流，相關流程可參考圖 5-1。

當網域內發生壅塞時，且壅塞的資料流都是(1)正常傳送的資料流，控制器就會從中挑選 Profit 值最大的資料流嘗試繞路或借路，這是因為 Profit 最高的資料流通常也相對重要，如果其他路徑有足夠的頻寬，就不會產生壅塞，而原網域內的其他資料流也得以獲得較好的收益，這部分請詳閱 5.3.1 節，然而，若是原網域內部沒有其他鏈路有足夠頻寬來支援問題流，則控制器會嘗試跟其他網域借用路徑，這部分請詳閱 5.3.2 節。

在有資料流借出到其他網域時，該網域(即協助網域)內就出現協助流，而當協助網域後來也發生壅塞時，協助網域會優先選擇協助流來進行限速，也就是情況(3)，這是因為協助流是幫忙性質的設計，對於協助網域本身相對來講不重要，所以優先對其進行限速，其中協助網域會透過章節 5.4 的 Nash Bargaining 模組計算要借出的頻寬，然後協助網域會發送 Rate\_limit\_Notify 的訊息給原網域，原網域會將資料流進行拆分，部分資料流持續透過協助網域借路來傳輸，剩餘的部分透過資料流原網域內的鏈路傳輸，這部分請參考章節 5.4，同時，若同時有多個協助流則會選擇 Profit  $\times$  Slice 值最低的資料流，Slice 值為在壅塞時原網域願意與



協助網域分享的部分(詳細說明可參考章節 5.4)，除此之外，若自身網域又出現鏈路壅塞的情況或是無法透過跨域其他網域協助減緩壅塞，則可再透過 5.5 章節的內容進行限速。

至於 $\sigma$ 數值的選擇，我們採用文獻[3]所使用的 20% 來當作門檻值，因其不會因為流量傳送時不穩定的抖動或是不大的封包丟失率就立即啟動負載平衡機制，以避免太頻繁或太早就嘗試把資料流引導到其他路徑上。

### 5.3.1 同網域的負載平衡

當某條鏈路出現壅塞情形時，若該鏈路上所承載的資料流皆屬於前面所描述的 (1) 正常傳送的資料流，亦即其來源與目的主機均位於該控制器所屬的網域中，其控制器將會針對其中 Profit 值最高的資料流進行繞路處理；路徑的重選機制與原始路徑的建立方式相似：首先，控制器會尋找所有可行的最短路徑，若有多條候選路徑，則進一步比較這些路徑的頻頸頻寬(Bottleneck bandwidth)，選擇其中剩餘頻寬最充足的一條，其中，頻頸頻寬指的是一條路徑上所有鏈路中，剩餘頻寬最少的那一條所能提供的頻寬量。

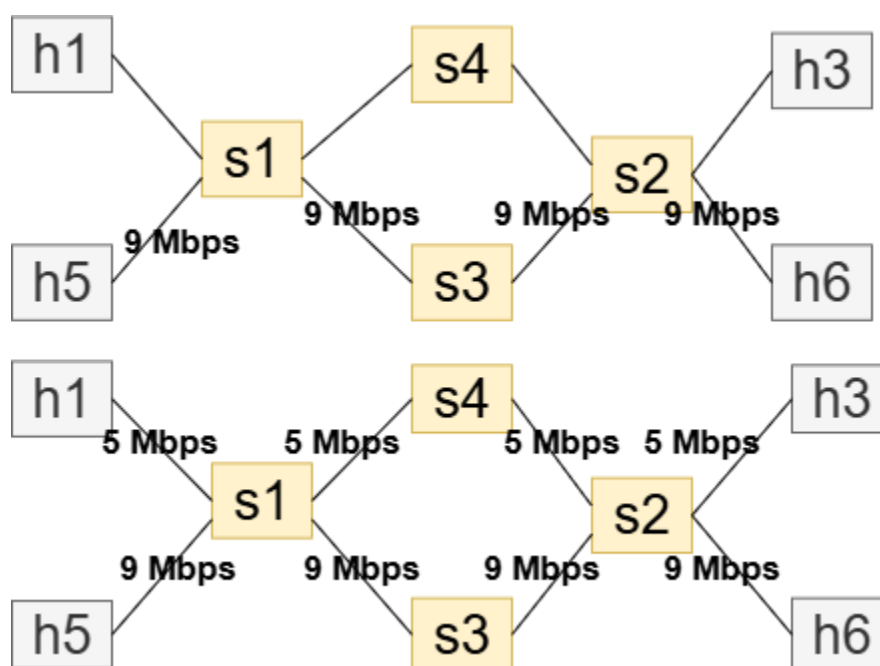


圖 5-3：內部負載平衡示意圖

以圖 5-4 為例，假設所有鏈路的初始頻寬均為 10 Mbps，當資料流  $h5 \rightarrow h6$  (其需求頻寬為 9 Mbps)，準備加入網路時，系統會發現兩條最短路徑可供選擇： $s1 \rightarrow s4 \rightarrow s2$  與  $s1 \rightarrow s3 \rightarrow s2$ ，由於這兩條路徑的頻頸頻寬皆為 10 Mbps，系統將隨機選擇其中一條，例如：選擇  $s1 \rightarrow s3 \rightarrow s2$ ，此時，該路徑上所有鏈路的可用頻寬將立即被更新：包括  $(h5, s1)$ 、 $(s1, s3)$ 、 $(s3, s2)$  及  $(s2, h6)$ ，皆剩下 1 Mbps。

接著，當另一條資料流  $h1 \rightarrow h3$  (需求頻寬為 5 Mbps) 嘗試加入時，系統再次尋找最短路徑，雖然可選擇的路徑仍為  $s1 \rightarrow s4 \rightarrow s2$  與  $s1 \rightarrow s3 \rightarrow s2$ ，但此時前者的頻頸頻寬已降至 1 Mbps，無法滿足需求，而後者仍維持 10 Mbps，因此  $h1 \rightarrow h3$  將選擇  $s1 \rightarrow s4 \rightarrow s2$  作為傳輸路徑。

透過此種基於路徑剩餘頻寬的選擇策略，能有效分散資料流於不同鏈路上，避免單一路徑發生過載，有助於降低整體網路壅塞的風險並提升傳輸效率。

### 5.3.2 跨網域負載平衡

倘若無法透過上述方式重新繞路來舒緩壅塞，控制器則會傳送 5.1 節提到的 Ask\_Help 訊息來請求其他網域的支援，而其他控制器則會判斷自身網域內是否有鏈路可以進行協助，若有則回覆 Can\_Help 訊息，反之則不予理會，而若請求協助的控制器收到多則 Can\_Help 訊息，控制器會選擇最先收到的訊息的網域進行繞路，但若是經過 Q 秒(各網域可自行進行設定)後仍然內沒有收到回覆，則代表其他網域無法進行支援，所以要對自身網域的流量進行限速以確保價值的產出，詳情請閱讀 5.5 節。

而若有協助網域回覆 Can\_Help 訊息，原網域就會將該資料流繞路到對應的 Gateway switch，以圖 5-4 為例，假設在網域 1 中，控制器選定資料流  $h1 \rightarrow h4$

為需借用外部路徑傳輸的協助流，可觀察到主機 h1 最接近的 Gateway switch 為 s1，而 h4 則連接至 s4。若該筆資料流的頻寬需求上限為 7 Mbps，且其 Profit 設定為 50，則控制器 C1 將發出一則 Ask\_Help 請求，其具體內容如下: Command = “Ask\_Help”，Domain = “1”，Out\_Sw = “S1”，In\_Sw = “S4”，Src\_IP = “10.0.0.1”，Dst\_IP = “10.0.0.4”，Bw = “7”，Profit= “50”，Slice= “0.3”。

收到此請求後，網域 2 的控制器 C2 會評估目前可行的跨域傳輸路徑，目前假設存在兩條備選路徑，分別為 s1 → s7 → s8 → s4 與 s1 → s7 → s5 → s6 → s8 → s4，這兩條路徑的頻頸頻寬分別為都是 10 Mbps，均可滿足此協助流的需求。此時，C2 將選擇最短且頻寬足夠的可行路徑給 C1，並傳送以下的 Can\_Help 訊息：Command = “Can\_Help”，Domain = “2”，In\_Sw = “S7”，Out\_Sw = “S8”，Src\_IP = “10.0.0.1”，Dst\_IP = “10.0.0.4”，透過這樣的跨網域協調機制，系統能夠靈活地調度可用路徑資源，以因應不同資料流的頻寬與優先權需求，並有效緩解來源網域的壅塞之狀況。

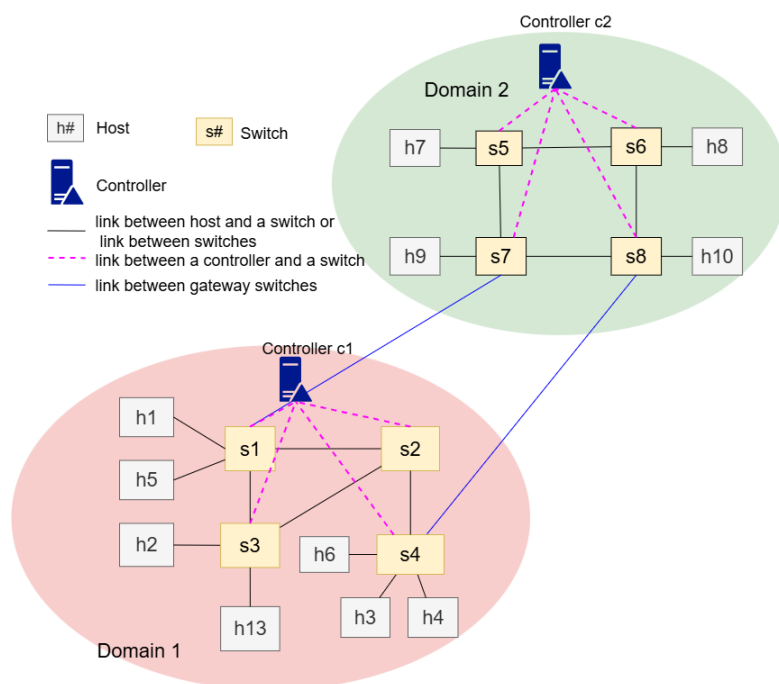


圖 5-4：跨網域協助傳輸之範例

## 5.4 Nash Bargaining 模組

當協助流進來協助網域後，協助網域可能會由於產生新的頻寬需求而導致鏈路發生壅塞，進而影響到協助網域本身的資料流，而文獻[3]中採用優先級等比例的方式分配頻寬給各資料流，相對地，本研究認為雖然各網域都為一個公司或是大型單位底下，它們沒有直接要提供協助的誘因，因此應加入一個額外的機制，使協助流的原網域分享部分 Profit，激勵網域間的互相協助。

至於實際決定分配頻寬多少的方式將透過此章節的 Nash Bargaining 模組來決定，這是個啟發於 Nash Bargaining Solution (NBS)的方式，類似的方式可以參考文獻[32][33]，其研究專注於在異構網域 (Heterogenous network domain)中對基地站的資源存取透過 NBS 得到較公平的資源分配方法，本研究的目的希望能設計一個由 NBS 啟發的方式，將 Profit 最大化的同時，也可避免資料流餓死，所以設計上會有所不同。

參考文獻[33]，本研究中的賽局描述如下：賽局中有  $N$  個玩家，每個玩家  $n (n \in \{1, 2, \dots, N\})$  要共享資源並使各自的功能函數 (Utility function) 的結果最大化，若玩家無法達成合作則會形成分歧點 (Disagreement point)，我們以  $D = (d_1, d_2, \dots, d_N)$  代表各玩家的分歧點的集合，其中，各玩家的分歧點表示當無法合作時各自玩家還能獲得的收益，另外，功能函數的集合為  $U = (u_1, u_2, u_3, \dots, u_N)$ ，其中  $U$  為  $R^N$  中一個封閉且凸 (Convex) 的子集合，表示在合作情況下所有參與者可達成的可行收益配置之集合；因為玩家最低能獲得的收益已被定義 (集合  $D$ )，所以集合  $\{u \in U \mid u_n \geq d_n, \forall n \in \{1, 2, \dots, N\}\}$  將為非空集合，此時 NBS 的解可以表示如下：

$$U^* = \arg \max_u \prod_{n=1}^N (u_n - d_n) \quad , s.t. \ u_n \geq d_n \quad (3)$$

本研究中會建立一個由兩個玩家組成的賽局，玩家 1 代表的是請求借鏈路的網域， $u_1$  表示如果借路成功依照借的頻寬量可以獲得的 Profit，玩家 2 代表協助網域， $u_2$  表示依照借出頻寬的多少玩家 2 整體可以得到的 Profit，在此 NBS 賽局中，本模組希望得到相較於談判失敗，能讓雙方功能函數的結果增加量的乘積最大的解，至於協商結果必須保證雙方至少都能獲得其分歧點(即  $d_n$ )或是以上的效益值。

賽局中的限制有鏈路的最大頻寬  $C_l$ ，以及每個資料流能分到的頻寬不可大於其要求的頻寬，且分配到的頻寬不可低於 0，即對於協助流而言， $0 \leq B_1^a \leq B_1^d$  ( $B_1^d$  為協助流要求的流量， $B_1^a$  為協助流被分配的流量)，在此賽局中，對於玩家 1 是相對不利的，因為對於玩家 2 (協助網域)而言，它沒有原因要協助玩家 1，有鑑於，本系統中加入價值分享機制 Slice (以參數  $s$  來表示)，來激勵玩家 2 提供協助，特別來講，玩家 1 提供部分 Profit ( $\bar{P}_1$ ) 作為回報，玩家 2 能得到的 Profit 可依照該資料流傳輸時長、協助網域在壅塞時協助傳輸的時長、協助流要求的頻寬  $B_1^d$ 、協助流得到的頻寬  $B_1^a$  來求得，其計算方式如下：

$$\text{協助網域獲得Profit} = \frac{\text{借路壅塞的傳輸時長}}{\text{資料流整體傳輸時長}} \times \frac{B_1^a}{B_1^d} \times \bar{P}_1 \times s \quad (4)$$

$$, s.t. \ 0 \leq s \leq 1$$

舉例來說，一個資料流從網域 1 到網域 2 借路，其需求頻寬為 5 Mbps，Profit 為 80，總傳送時長(包含還沒繞路到網域 2 的時間與到網域 2 借路但是還沒壅塞的期間)為 100 秒，網域 2 在借路的鏈路壅塞且願意提供 3Mbps 的頻寬，它提供協助 50 秒，網域 1 提供的 Slice 值為 0.3，那麼協助網域獲得 Profit 為

$$\frac{50}{100} \times \frac{3}{5} \times 80 \times 0.3 = 7.2。$$

基於以上，玩家 1 的功能函數如下：

$$u_1 = (1 - s) \times \frac{B_1^a}{B_1^d} \times \bar{P}_1, \quad (5)$$

$$s.t. \ 0 \leq B_1^a \leq B_1^d, \ 0 \leq s \leq 1$$

玩家 1 的分歧點為：

$$d_1 = 0 \quad (6)$$

這是因為如果不合作，玩家 1 無法透過協助網域獲得任何 Profit。

另一方面，若是玩家 2 提供頻寬給玩家 1，則有可能會犧牲自身網域資料流的價值，因此在衡量功能函數時，玩家 2 單純考慮自己能透過協助所能獲得的價值，而為簡化計算過程，玩家 2 會先統計自己網域中資料流的要求頻寬與他們的 Profit，並當成一整個資料流來在賽局中與玩家 1 談判，協助網域自身資料流的所有需求頻寬定義為  $B_2^d = \sum_{k=1}^K b_k^d$ ，K 為協助網域中的所有資料流的合集，協助網域自身資料流的所有價值定義為  $\bar{P}_2 = \sum_{k=1}^K P_k$ ，則玩家 2 功能函數定義如下：

$$\begin{aligned} u_2 &= \bar{P}_2 \times \frac{\min\{C_l - B_1^a, B_2^d\}}{B_2^d} + s \times \frac{B_1^a}{B_1^d}, \\ s.t. & 0 \leq B_1^a \leq B_1^d, 0 \leq s \leq 1 \end{aligned} \quad (7)$$

玩家 2 的分歧點為：

$$d_2 = \bar{P}_2 \times \min \left\{ \frac{C_l}{B_2^d}, 1 \right\} \quad (8)$$

這是因為若沒有合作，則玩家 2 僅可以獲得沒有協助時可獲得的 Profit，也就是完整  $\bar{P}_2$  與鏈路最大頻寬能夠滿足的自身需求的比例。

結合公式(3)、(5)、(6)、(7)、(8)，此賽局的目標如下：

$$\begin{aligned} U^* &= \arg \max_u \left\{ \left[ (1-s) \times \frac{B_1^a}{B_1^d} \times \bar{P}_1 \right] \right. \\ &\times \left[ \bar{P}_2 \times \frac{\min\{C_l - B_1^a, B_2^d\}}{B_2^d} + s \times \frac{B_1^a}{B_1^d} - \bar{P}_2 \times \min \left\{ \frac{C_l}{B_2^d}, 1 \right\} \right] \left. \right\} \\ &, s.t. 0 \leq B_1^a \leq B_1^d, 0 \leq s \leq 1 \end{aligned} \quad (9)$$

本研究在此模組中使用 Python 的 SciPy 套件所提供之 scipy.optimize 模組，並採用 minimize\_scalar 函數來求解單一變數的最大化問題[34]，本模組僅需針對協助流的頻寬進行調整，且變數僅有一個(即 s)，minimize\_scalar 函數適用於此類一維函數的優化情境，並可透過定義上下限的方式，確保解位在合理範圍內，同時，由於 minimize\_scalar 為最小化工具，我們將原本的目標函數乘以負號以轉換為等

效的最小化形式，求解結果即為最適合的協助頻寬配置，達成雙方建立在 Profit 之上的頻寬分配。

| 參數                                | 意義                                   |
|-----------------------------------|--------------------------------------|
| $U = (u_1, u_2, u_3, \dots, u_N)$ | 各玩家功能函數的集合                           |
| $D = (d_1, d_2, \dots, d_N)$      | 各玩家分歧點的集合，N 為玩家數量，n 表示某 1 玩家。        |
| $u_1$                             | 玩家 1 的功能函數，代表被協助網域透過協商能獲取的 Profit。   |
| $u_2$                             | 玩家 2 的功能函數，代表協助網域透過協商能獲取的 Profit。    |
| $d_1$                             | 玩家 1 的分歧點，代表被協助網域若不能被協助所能獲取的 Profit。 |
| $d_2$                             | 玩家 2 的分歧點，代表協助網域若不提供協助能獲取的 Profit。   |
| $C_l$                             | 壅塞鏈路的最大頻寬                            |
| $B_1^a$                           | 協助流被分配的頻寬                            |
| $B_1^d$                           | 協助流要求的頻寬                             |
| $B_2^d$                           | 在壅塞鏈路上，協助網域中自身 (不包含協助流) 的資料流總共所需要的頻寬 |
| $\bar{P}_1$                       | 協助流的價值                               |
| $\bar{P}_2$                       | 在壅塞鏈路上，協助網域自身資料流總共的價值                |
| s                                 | Slice 參數，表示協助流願意分享的價值量。              |

表 5-7：模組中各參數所代表的意義

## 5.5 Profit Maximization 模組

本研究希望最大化獲得的 Profit 並避免資料流 Starvation，這可透過本章節的 Profit Maximization 模組來實驗，詳細參數表可參考表 5-8，其中，此系統的計算

方式是使用資料流的封包丟失率來計算獲得的 Profit，直觀的計算方式如下：

$$G_i = P_i \times \frac{b_i^a}{b_i^d} \quad (10)$$

資料流  $f_i$  所獲得的 Profit gain  $G_i$  為該資料流的 Profit 乘上以資料流成功送達的比率，可以用  $f_i$  所獲得的頻寬除上  $f_i$  所需要的頻寬來估算，由於無法得知一條資料流會傳輸多久，本模組會嘗試最大化當下可以獲得的價值。

為進一步在此系統中實作，本模組設計一個功能函數，在進行流量限速時要將此功能函數所獲得的值最大化，同時為避免部分資料流完全無法獲得頻寬，導致餓死，本模組還加入懲罰機制：

$$\max \sum_{\forall f_i \in F}^N \left( G_i - \frac{b_i^d - b_i^a}{\max(b_i^a, \varepsilon) \times P_i} \right), \text{ s.t. } 0 \leq b_i^a \leq b_i^d \quad (11)$$

以下針對功能函數進行描述，求和符號內的第一個項目  $G_i$  為資料流的價值獲取估算，所有資料流價值加總即為目前配置下的整體效益表現，系統的第一個目標就是將其最大化。求和符號內的第二項  $\frac{b_i^d - b_i^a}{\max(b_i^a, \varepsilon) \times P_i}$  的目的是進行未被滿足之流量的補償性懲罰，對弱勢流量進行資源保障，在最大化獲取價值的同時，往往會使低價值的資料流無法取得任何流量，當某個低的流量被完全忽略 ( $b_i^a \rightarrow 0$ )，本懲罰值趨近無限大，系統效用嚴重下降，這會迫使資源配置策略重新考量這些被犧牲的流量，同時分母多採用一項資料流價值  $P_i$  作為限制參數，若該資料流價值越大將使懲罰越小，以進一步提升高價值資料流的頻寬，除此之外，我們採用  $b_i^d - b_i^a$  當作分子，這是因為當此值越大，代表此資料流越嚴重未被滿足，結合在一起，逞罰函數為資料流不被滿足的頻寬除以資料流被分配的頻寬與它的價值，當分配頻寬趨近於 0，則懲罰越大，當沒被滿足的頻寬越高，懲罰越大，當資料流價值越大，那懲罰越小，此外，公式(11)中的  $\varepsilon$  代表極小值，例如  $10^{-10}$ ，能有效避免在功能函數最佳化的過程中整除 0，以及在配置頻寬為 0 時給予有意義的懲罰。



| 參數            | 解釋                                    |
|---------------|---------------------------------------|
| $P_i$         | Flow $f_i$ 沒有丟失封包，完全傳送的情況下能獲得的 Profit |
| $b_i^a$       | Flow $f_i$ 預估被分配的頻寬                   |
| $b_i^d$       | Flow $f_i$ 所需要的頻寬                     |
| $G_i$         | 估算 Flow $f_i$ 所獲得的 Profit Gain $G_i$  |
| $\varepsilon$ | 極小數，在本模組中用來避免整除 0 的情況發生               |

表 5-8：Profit Maximization 模組之參數表

我們採用 Python 中的 SciPy 套件進行功能函數的最佳化求解，由於此問題涉及多個條件限制與變數關係，因此選用 scipy.optimize 模組中的 minimize 函數[31]來處理具有約束條件的最佳化問題，另外，有於 minimize 函數會去找尋最小化的解，因此我們需將原本的最大化目標函數進行符號反轉，將其轉換為等效的最小化形式，而在演算法的選擇上，我們採用 Sequential Least Squares Programming (SLSQP)方法，該方法能處理含有不等式與等式限制的問題，適合我們針對每條資料流所設定的頻寬需求、實際配置值、價值與可用資源上限等多重條件下，進行具體的數值求解，相關限制如下：

- 1) 配置給資料流的頻寬不可超過該資料流需要的頻寬：

$$0 \leq b_i^a \leq b_i^d \quad (12)$$

- 2) 所有配置的頻寬加起來不會超過鏈路的上限頻寬

$$\sum_{\forall f_i \in F}^N (b_i^a) \leq C_l \quad (13)$$

在本研究中，PMM 與 NBM 均以連續函數 (Continuous function)定義，並且有明確的邊界條件(例如：總頻寬容量限制、分配值非負等)，依據「極值定理」(Extreme value theorem) [42]，連續函數在有限閉區間上必然存在最小值或最大值，因此演算法不會陷入死結 (Deadlock)，此外，方法們所採用的

scipy.optimize.minimize 與 minimize\_scalar 方法均為保證收斂的數值演算法，確保系統一定能得到一個解而不會卡住，也就是不會形成死結。

以下對 PMM 進行舉例，有三條要進行限速的資料流，詳情見表 5-9、表 5-10、表 5-11，第一組資料流 Profit 與頻寬大小不相關，第二組頻寬要求與 Profit 正相關，第三組頻寬要求與 Profit 負相關，假設所有資料流開始與結束的時間都一樣，鏈路最大頻寬為 10 Mbps，這三組數據中包含五條資料流以及它們要求的頻寬(BW)和對應的 Profit，並且分別顯示了使用本模組進行限速以及使用 Greedy 方式進行限速分別會得到的結果，其中，Greedy 方法是指優先滿足單位價值最高的資料流，也就是 Profit 除 BW 最高的資料流(可以想成是每分配 1 Mbps 給該資料流可以獲得多少 Profit)，若有資料流 Profit/BW 值一樣則會從 BW 小的優先滿足，這個方法會在鏈路壅塞情況下最大化得到的 Profit，三個範例中我們可以發現 Greedy 方法會造成部分資料流餓死，而本模組可以避免資料流餓死且達成接近 Greedy 的 Profit gain，特別來說，第一組 Greedy 的 Profit gain 為 110，本模組的 Profit Gain 為 91.5733，差了 16%；第二組 Greedy 的 Profit gain 為 100，本模組的 Profit Gain 為 97.4944，只差 2.5%；第三組 Greedy 的 Profit gain 為 210，本模組的 Profit Gain 為 204.65，只差 2.5%；另外，第一組中 Greedy 會造成 3 個資料流餓死，而第二組與第三組中使用 Greedy 方法都會造成 1 個資料流餓死，第一組中為了避免三個資料流餓死所以損失的價值相對多，但也形成了一個比較平衡的系統。

| Case1：不相關 |    |        | profit_maximize_module |        |         | Greedy        |         |      |
|-----------|----|--------|------------------------|--------|---------|---------------|---------|------|
| Flow      | BW | Profit | Alloc                  | Loss%  | Gain    | Alloc         | Loss%   | Gain |
| 1         | 8  | 10     | 0.8847                 | 88.94% | 1.1059  | 0             | 100.00% | 0    |
| 2         | 8  | 90     | 6.0222                 | 24.72% | 67.7503 | 8             | 0.00%   | 90   |
| 3         | 5  | 40     | 1.2002                 | 76.00% | 9.6017  | 0             | 100.00% | 0    |
| 4         | 5  | 10     | 0.7266                 | 85.47% | 1.4532  | 0             | 100.00% | 0    |
| 5         | 2  | 20     | 1.1662                 | 41.69% | 11.6623 | 2             | 0.00%   | 20   |
|           |    |        | Total Profit:          |        | 91.5733 | Total Profit: |         | 110  |

表 5-9：頻寬要求與 Profit 不相關

| Case2：正相關 |    |        | profit_maximize_module |        |         | Greedy        |         |      |
|-----------|----|--------|------------------------|--------|---------|---------------|---------|------|
| Flow      | BW | Profit | Alloc                  | Loss%  | Gain    | Alloc         | Loss%   | Gain |
| 1         | 9  | 80     | 2.2551                 | 74.94% | 20.0452 | 0             | 100.00% | 0    |
| 2         | 6  | 60     | 2.9575                 | 50.71% | 29.5754 | 4             | 33.33%  | 40   |
| 3         | 3  | 30     | 2.0723                 | 30.92% | 20.7234 | 3             | 0.00%   | 30   |
| 4         | 2  | 20     | 1.715                  | 14.25% | 17.1503 | 2             | 0.00%   | 20   |
| 5         | 1  | 10     | 1                      | 0.00%  | 10      | 1             | 0.00%   | 10   |
|           |    |        | Total Profit:          |        | 97.4944 | Total Profit: |         | 100  |

表 5-10：頻寬要求與 Profit 正相關

| Case3：負相關 |    |        | profit_maximize_module |        |          | Greedy        |         |      |
|-----------|----|--------|------------------------|--------|----------|---------------|---------|------|
| Flow      | BW | Profit | Alloc                  | Loss%  | Gain     | Alloc         | Loss%   | Gain |
| 1         | 9  | 10     | 1.3751                 | 84.72% | 1.5279   | 0             | 100.00% | 0    |
| 2         | 6  | 30     | 2.6249                 | 56.25% | 13.1244  | 4             | 33.33%  | 20   |
| 3         | 3  | 50     | 3                      | 0.00%  | 50       | 3             | 0.00%   | 50   |
| 4         | 2  | 60     | 2                      | 0.00%  | 60       | 2             | 0.00%   | 60   |
| 5         | 1  | 80     | 1                      | 0.00%  | 80       | 1             | 0.00%   | 80   |
|           |    |        | Total Profit:          |        | 204.6523 | Total Profit: |         | 210  |

表 5-11：頻寬要求與 Profit 負相關

## 第六章 效能評估

### 6.1 實驗環境與工具

本章節透過模擬的方式來驗證我們所提出的 PNCFM 方法之效能表現，模擬中使用工具與其版本詳列於表 6-1，整體系統實作可分為四個模組，包括：控制器、網路拓撲建構、交換器設定，與流量生成等。在環境設置方面，本研究於 Ubuntu 20.04 系統下採用 Ryu [10] 作為 SDN 控制器，並透過 Mininet [35] 建立模擬網路環境，為能支援 Meter table 控制功能，我們所使用的 Open vSwitch 版本需為 2.8.1 以上，並搭配 OpenFlow 1.3 通訊協定來進行實作。

此外，由於 Open vSwitch 預設的 Group table 轉發邏輯會依據目標節點的 MAC 位址進行 Hash 運算，並非隨機選擇，因此本研究參考文獻[36]的方式修改其封包轉發流程，以實現機率式隨機轉發功能，在流量產生方面，我們使用 Iperf [36] 工具來建立 UDP 資料流，每個封包大小為 1470 bytes，用以模擬實際網路中不同行為的資料流需求。

| 模擬工具         | 版本      |
|--------------|---------|
| Ubuntu       | 20.04   |
| Ryu          | 3.24    |
| Open vSwitch | 3.5.0   |
| Mininet      | 2.3.0d6 |
| iperf        | 2.0.5   |
| OpenFlow     | 1.3     |

表 6-1：模擬工具與其使用版本

## 6.2 網路拓樸與流量配置

本實驗中所設計的網路拓樸分為三個獨立的網域，每個網域皆由一個專屬的控制器來負責管理，如圖 6-1 所示，交換器的角色可分為兩類：一為各網域內部所屬的交換器，另一則為連接不同網域之間的 Gateway switch，拓樸中的每條連接鏈路均設定為 10 Mbps 的頻寬上限。

而在傳輸速率的控制方面，本研究將鏈路的流量限制機制改為 Token Bucket Filter (TBF)，以避免速率變動所造成的模擬誤差，並可確保封包遺失率與實際網路負載更為一致，此外，TBF 同時他設計相對簡單，封包壅塞時會採 First-in-first-out (FIFO)策略，對於不同重要性的封包較沒有影響，可以更好展現此研究中壅塞管理機制之表現。

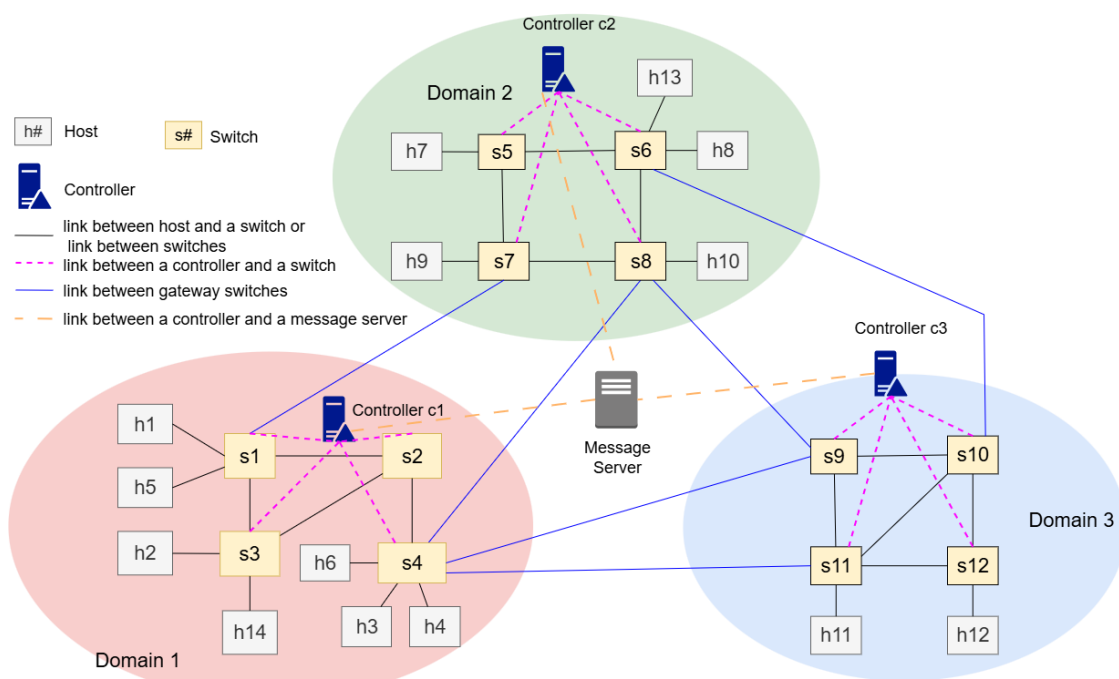


圖 6-1：實驗所使用的網路拓樸圖

我們將本論文所提出的 PNCFM 方法與三種方法進行比較，分別說明如下：

1. Baseline (無跨域協助)：此為基準方法，它不採取跨網域的路由策略，當某個 Domain 發生壅塞時，該網域的控制器僅會嘗試在本地鏈路中重新尋找閒置路徑，並無法透過其他網域進行協助，而若是無其他鏈路則不會對資料流再進行頻寬分配。
2. Cooperative flow management (CFM) [38]：此方法允許控制器向相鄰網域借用路徑來協助流量傳輸，然而控制器間並未共享網域相關資訊，亦未考慮到資料流的價值或是重要性之差異。
3. Multi-domain collaborative route management (MCRM) [3]：此方法在允許跨域合作的同時，考量到資料流權重，原作中採用優先級(Priority)並考量資料流的異質性，而在本實驗中則改為使用 Profit 針對不同流量需求進行差異化管理與資源分配，特別來說，原作的協助流在協助網域時優先級會減二，但由於 Profit 機制相對不同，在此研究中會以協助流原 Profit 乘上 0.50 以進行模擬，其原因是原作中的模擬範例中 Priority 的範圍為[1,5]，每減二就會損失約 50%的重要性。

我們將實驗分為 2 部分，實驗 1 為整體效能的評估，觀察各方式在不同壅塞情境的表現，至於實驗 2 則著重於不同 Profit 與頻寬需求的相關性之分析。

## 6.3 實驗 1：整體效能評估

在第一組實驗當中，我們在網路環境中建立多條資料流，涵蓋網域內部傳輸與跨網域的通訊情境，其中，跨域資料流指的是來源端與目的端分別位於不同的網域，而為了方便說明各情況，本實驗將不同情境列出，對於各情境的資料流起始與結束時間分別如圖 6-2、圖 6-3、圖 6-4、圖 6-5 所示，每個實驗的長度皆設為 250 秒，詳細的流量資訊則列於表 6-2，其中單位 Profit 代表各資料流每完整傳送 1 Mbps 的頻寬可以帶來的 Profit 值，以下針對各情境進行說明分別對應以下時

段：

- 1) Case 1：原網域發生壅塞時，CFM、MCRM、PNCFM 都把封包導到協助網域，而協助網域無壅塞可以傳輸，Baseline 不進行跨域合作或是壅塞管理。
- 2) Case 2：原網域壅塞發生，同時協助網域也壅塞，而無法將資料流導到協助網域，CFM、MCRM、PNCFM 都會對各自網域內的資料流進行限速。
- 3) Case 3：原網域發生壅塞，CFM、MCRM、PNCFM 都把封包導到協助網域，然而，後續協助網域發生壅塞，MCRM 與 PNCFM 會把部分流量導回原網域，但 CFM 未另外處理，協助流於協助網域也沒有被另外處理。
- 4) Case 4：在 Case 3 的基礎上，後續自身網域也壅塞，此時 MCRM 並沒有另外對回來後的資料流進行限速，至於 PNCFM 有另外對資料流進行限速來提升 Profit。

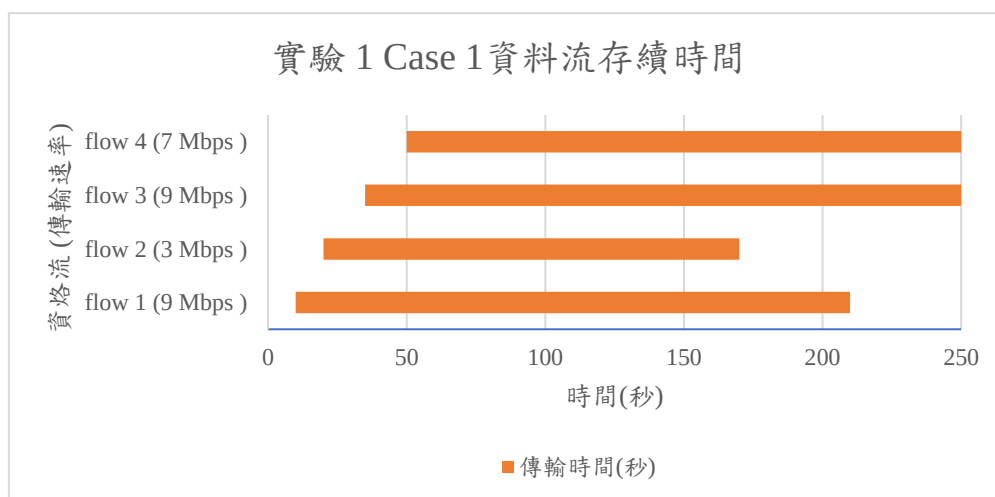


圖 6-2：實驗 1 Case 1 資料流存續時間

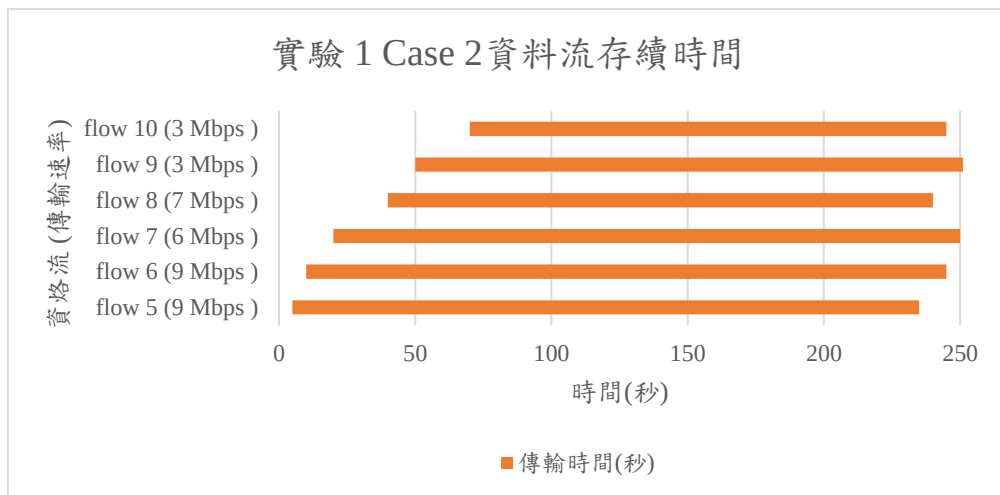


圖 6-3：實驗 1 Case 2 資料流存續時間

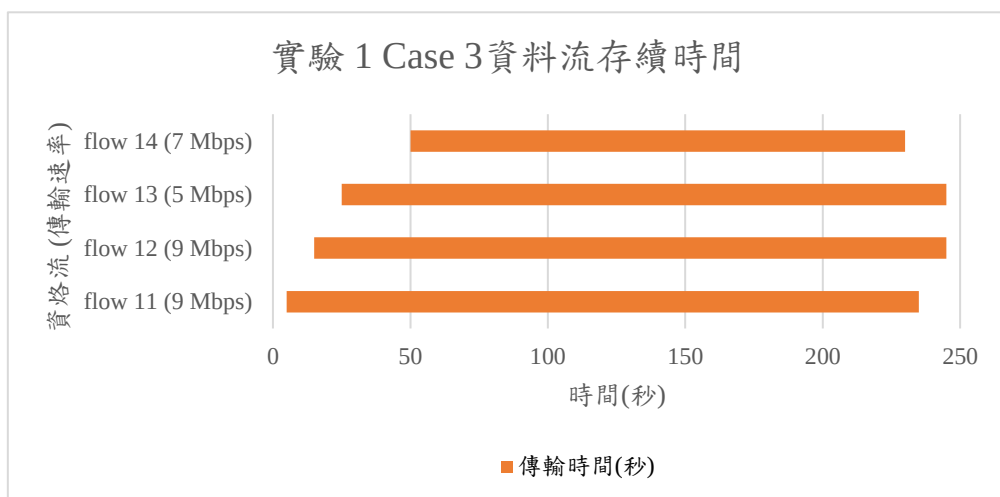


圖 6-4：實驗 1 Case 3 資料流存續時間

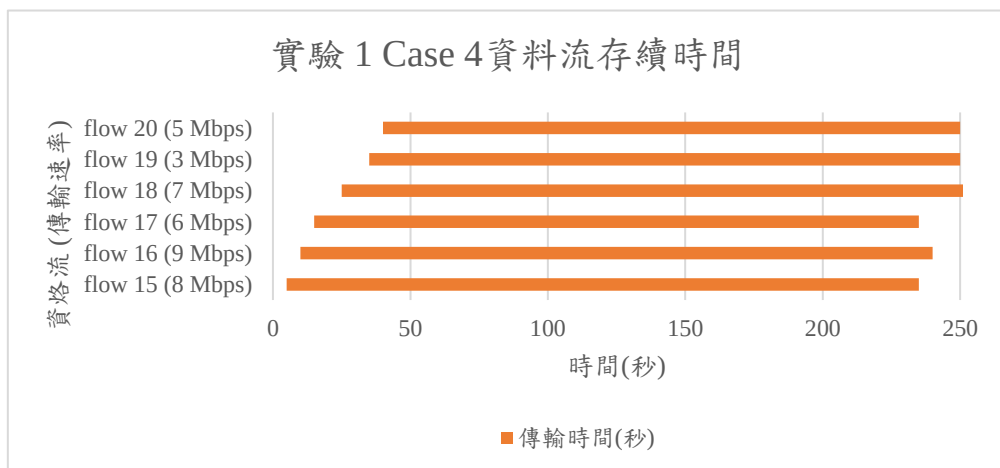


圖 6-5：實驗 1 Case 4 資料流存續時間



| Case   | 起始主機 | 目標主機 | 資料流     | 頻寬 (Mbps) | 單位 Profit | 總 Profit | 起始時間 | 結束秒數 | 傳輸時間 | 跨網域 |
|--------|------|------|---------|-----------|-----------|----------|------|------|------|-----|
| Case 1 | h4   | h2   | flow 1  | 9         | 7         | 63       | 10   | 210  | 200  | ✓   |
|        | h10  | h12  | flow 2  | 3         | 10        | 30       | 20   | 170  | 150  | ✓   |
|        | h3   | h1   | flow 3  | 9         | 2         | 18       | 35   | 250  | 215  |     |
|        | h7   | h8   | flow 4  | 7         | 3         | 21       | 50   | 250  | 200  |     |
| Case 2 | h7   | h8   | flow 5  | 9         | 5         | 45       | 5    | 235  | 230  |     |
|        | h10  | h9   | flow 6  | 9         | 2         | 18       | 10   | 245  | 235  |     |
|        | h5   | h6   | flow 7  | 6         | 4         | 24       | 20   | 250  | 230  |     |
|        | h2   | h3   | flow 8  | 7         | 2         | 14       | 40   | 240  | 200  |     |
|        | h14  | h3   | flow 9  | 3         | 10        | 30       | 50   | 260  | 210  |     |
|        | h4   | h12  | flow 10 | 3         | 3         | 9        | 70   | 245  | 175  | ✓   |
| Case 3 | h1   | h6   | flow 11 | 9         | 8         | 72       | 5    | 235  | 230  |     |
|        | h9   | h10  | flow 12 | 9         | 5         | 45       | 15   | 245  | 230  |     |
|        | h2   | h3   | flow 13 | 5         | 2         | 10       | 25   | 245  | 220  | ✓   |
|        | h7   | h13  | flow 14 | 7         | 5         | 35       | 50   | 230  | 180  |     |
| Case 4 | h1   | h6   | flow 15 | 8         | 7         | 56       | 5    | 235  | 230  | ✓   |
|        | h9   | h10  | flow 16 | 9         | 3         | 27       | 10   | 240  | 230  |     |
|        | h2   | h3   | flow 17 | 6         | 2         | 12       | 15   | 235  | 220  |     |
|        | h7   | h13  | flow 18 | 7         | 7         | 49       | 25   | 255  | 230  |     |
|        | h5   | h4   | flow 19 | 3         | 2         | 6        | 35   | 250  | 215  |     |
|        | h14  | h3   | flow 20 | 5         | 10        | 50       | 40   | 250  | 210  |     |

表 6-2：實驗 1 的資料流設定

### 6.3.1 Throughput 分析

情境一到情境四的吞吐量結果分別於圖 6-6、圖 6-7、圖 6-8、圖 6-9 顯示，在這四個情境中，若有壅塞發生，都是網域 1 發生壅塞，此外 CFM、MCRM、PNCFM 等方法會讓網域 1 的控制器嘗試將資料流導到網域 2，即協助網域。

情境一中由於 Baseline 無法將壅塞節點的資料流導到協助網域，所以吞吐量表現最差，CFM、MCRM、PNCFM 三個方法都可以在本地網域壅塞時，順利將資料流借路到另一個網域並提升整體吞吐量；而在情境二當中，CFM、MCRM、PNCFM 三個方式法嘗試在自己網域壅塞時將資料流導到協助網域，但是由於沒有協助網域可以提供協助，MCRM 與 PNCFM 都依照自己的方法在自己的網域內進行限速，至於 CFM 與 Baseline 沒有另外設計依照資料流重要性的限速機制，

四個方式的整體吞吐量因為達到上限所以是一樣的，而在第三個情境中，原網域(網域 1) 發生壅塞，CFM、MCRM、PNCFM 都把封包導到協助網域(網域 2)，後續協助網域在第 25 秒 h7 到 h13 的資料流啟動後發生壅塞，MCRM 與 PNCFM 會把部分流量透過原網域傳輸，因此吞吐量表現較好，而 CFM 不會將資料流導回原網域，造成在此情況時 CFM 的吞吐量表現會比什麼都不做的 Baseline 還差，除此之外，我們可以於第 35 秒到第 220 秒的時間觀察到 PNCFM 的 Throughput 比 MCRM 相對高一點，這是因為 MCRM 會依照等比例分配頻寬的方式會犧牲網域二部分的資料流的頻寬，而 PNCFM 相較於 MCRM 會拆分較多頻寬來不透過網域 2 進行傳輸，因而提升整體吞吐量。

在第四個情境中，在網域 1 會發生壅塞，CFM、MCRM、PNCFM 三個方法都選擇將 h1 到 h6 的資料流當成要處理的資料流，因它的 Profit 是壅塞節點中的資料流內最高的，然後將其導到網域 2 (協助網域)，而在那階段網域 2 是可以支援並沒有壅塞，所以後續 h7 到 h13 資料流啟動形成壅塞，但 CFM 沒有另外將資料流進行處理，MCRM 與 PNCFM 都將部分 flow 15 (即 h1 到 h6 的資料流)導回原網域進行傳輸，所以吞吐量都比 CFM 來得高，其中，MCRM 中的協助網域採用 Profit 等比例頻寬分配的方式決定要給 h1 到 h6 的資料流多少頻寬，PNCFM 則採用 5.4 節的 Nash Bargaining 模組，但整體吞吐量變化不大，特別來說，於時間 35 秒前 flow 19 (h5 到 h4 的資料流)未啟動，CFM 中網域 1 的 flow 15 的流量會在網域 2 傳輸，MCRM 與 PNCFM 會拆分資料流使部分透過原網域傳輸，使得原網域部分鏈路接近壅塞，而在時間 35 秒時，flow 19 資料流啟動、40 秒時 flow 20 (即 h14 到 h3 的資料流) 啟動後，CFM 則可以利用還有空間的鏈路來提升吞吐量，但是 MCRM 與 PNCFM 的網域 1 與網域 2 都產生壅塞，這兩個方法的吞吐量無法再進一步提升，順帶一提，Baseline 因為無法進行跨域合作，所以整體吞吐量表現都較低。

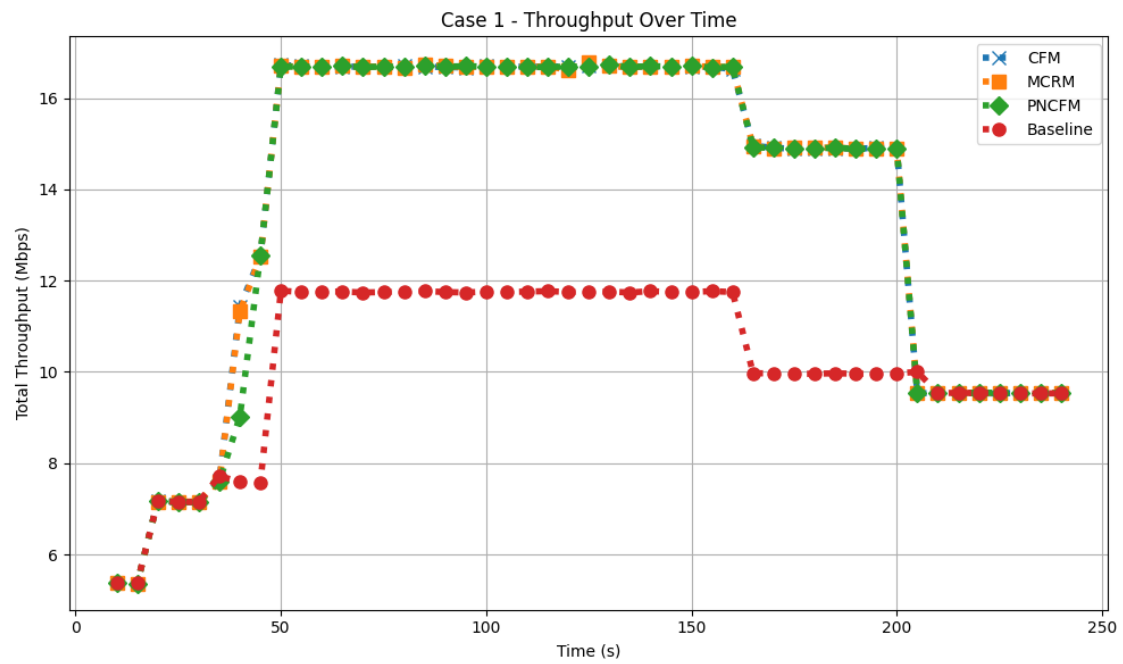


圖 6-6：實驗 1 Case 1 之整體系統吞吐量

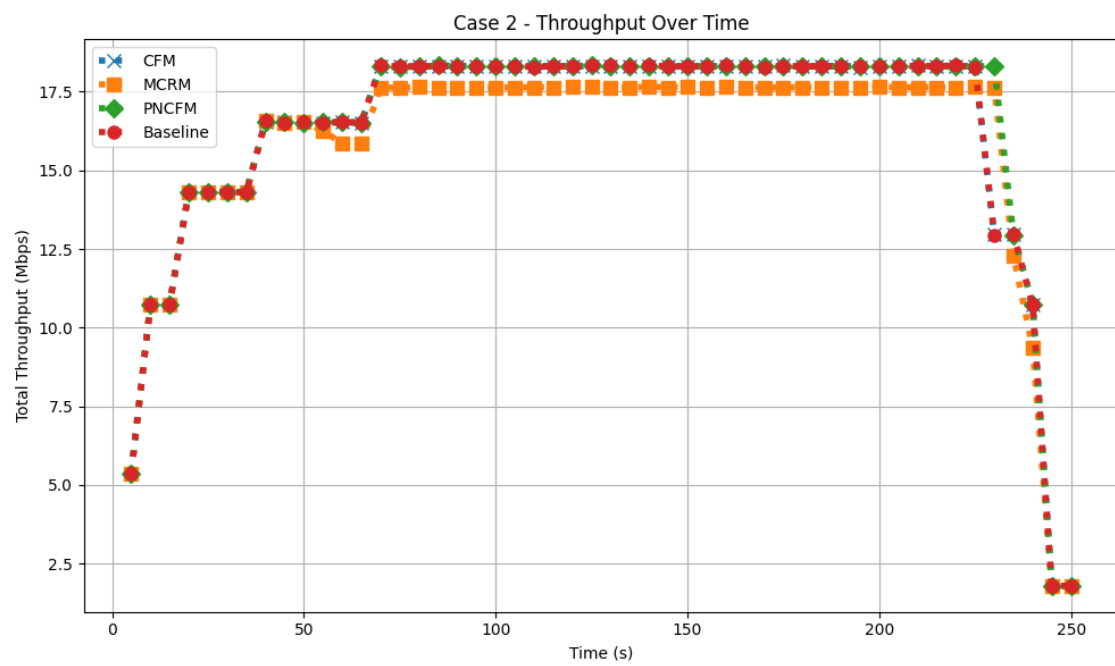


圖 6-7：實驗 1 Case 2 之整體系統吞吐量

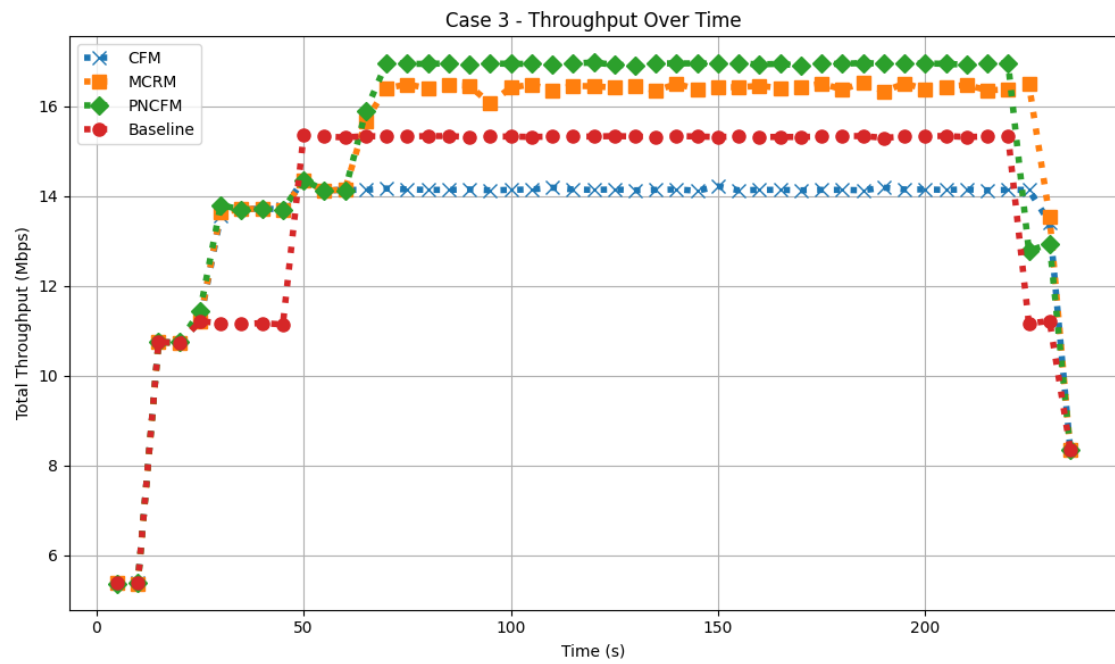


圖 6-8：實驗 1 Case 3 之整體系統吞吐量

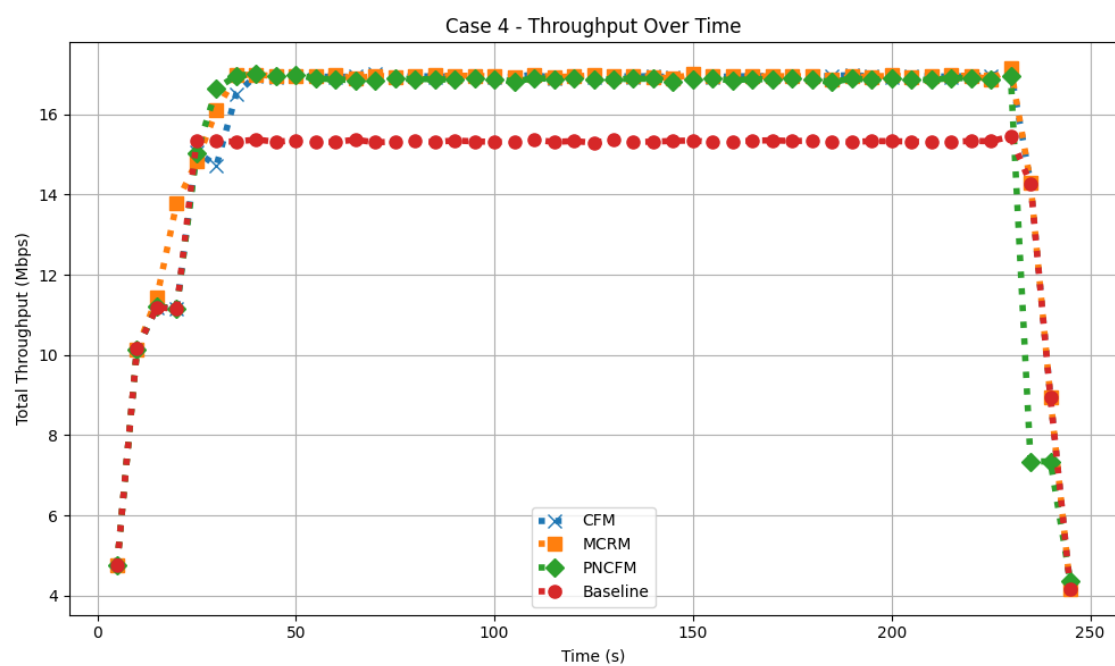


圖 6-9：實驗 1 Case 4 之整體系統吞吐量

### 6.3.2 獲取 Profit 分析

接著我們針對四個情境中所獲得的 Profit 進行分析，Profit 獲取是依照資料流整體的封包丟失率來決定的，所以 Profit 獲取可以由封包丟失率計算而來，情境一到四的相關結果與數據可以看查表 6-3，相關圖表則可參考圖 6-10、圖 6-11、圖 6-12、圖 6-13，這 4 個實驗情境中，網域 1 為壅塞資料流的原始網域，網域 2 為提供協助的協助網域，實驗結果顯示，在情境一中，CFM、MCRM、PNCFM 都成功在網域 1 壅塞時將資料流導到協助網域來舒緩壅塞，所以他們表現都比 Baseline 來得好，且後續沒有出現其他壅塞情況，所以 3 個方法所得到的 Profit 都會近似。

在情境二中，網域 1 發生壅塞，其控制器會請求協助，但是由於網域 2 已有其他資料流在鏈路上，沒有足夠頻寬提供協助，因此所有方法們會啟動各自的壅塞管理方式，Baseline 與 CFM 無另外設計壅塞管理方式，MCRM 會依照 Profit 等比例分配頻寬，至於 PNCFM 會依照章節 5.5 的 PMM 分配頻寬，由實驗結果可見，由於 PNCFM 會嘗試最大化 Profit，所以它會表現會是最好。

在情境三當中，原網域發生壅塞，CFM、MCRM、PNCFM 都會把封包導到協助網域，但後續協助網域發生壅塞，MCRM 與 PNCFM 把部分流量導回原網域，至於 CFM 則未另外處理資料流，協助流因此被完整傳到協助網域造成協助網域壅塞，Baseline 沒有跨域合作或是壅塞管理，但是不會造成協助網域的壅塞，所以表現比 CFM 來得好，而 CFM 因為將 flow 11 (即 h1 到 h6 的資料流)導到網域 2 在網域二形成壅塞，所以表現最差；另一方面，MCRM 用 Profit 比例決定要借給協助流的頻寬，並將剩餘資料流透過原始網域傳輸，但其結果犧牲部分 flow 14 (即 h7 到 h13 的資料流) 的頻寬，圖 6-12 中可看到 MCRM 於 flow 14 價值只有獲得 26，相較於其他方式所獲得的 35，相較之下，PNCFM 是透過 5.4 節的 NBM 方式決定要借給協助流的頻寬，因協助流提供給協助網域的 Profit 不夠高，並沒有犧牲 flow 14 資料流，整體吞吐量也相對處於高點，因此 PNCFM 的方式比

MCRM 的方式產生更多 Profit，同時也是四個方法中表現最好的。

情境四是實驗一中造成最多壅塞的情境，特別來說，最壅塞的節點上同時有 4 條資料流在互相競爭資源，分別為 flow 15 (h1 到 h6)、flow 17 (h2 到 h3), flow 19 (h5 到 h4), flow 20 (h14 到 h3)，而在 25 秒，flow 18 (h7 到 h13) 啟動後，MCRM 與 PNCFM 會將部分資料流拆分並透過原網域傳輸，後來 35 秒時 flow 19 啟動、40 秒時 flow 20 啟動，使得網域變的壅塞且這兩個方式都無法進一步借路，其中只有 PNCFM 會對此情況做提升 Profit 的壅塞管理方式來進行頻寬分配，而 Baseline 無壅塞管理或是借路，因此表現最差，CFM 將一個資料流繞道協助網域傳輸，但後來沒有其他機制，因此表現第二差，MCRM 在將部分協助流拆分會來後未進行進一步的壅塞管理，表現第二好，至於 PNCFM 最完整表現最好，由表 6-4 結果而知，我們可以發現 PNCFM 比 Baseline 的 Profit 多得到 42%，比 CFM 多得 22%，比 MCRM 多得到 11%。

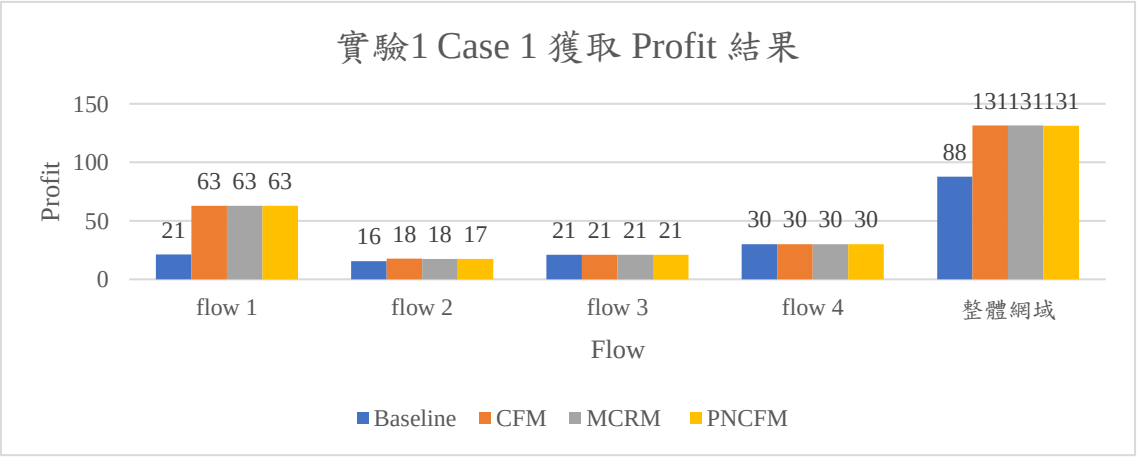


圖 6-10：實驗 1 Case 1 所獲取的 Profit

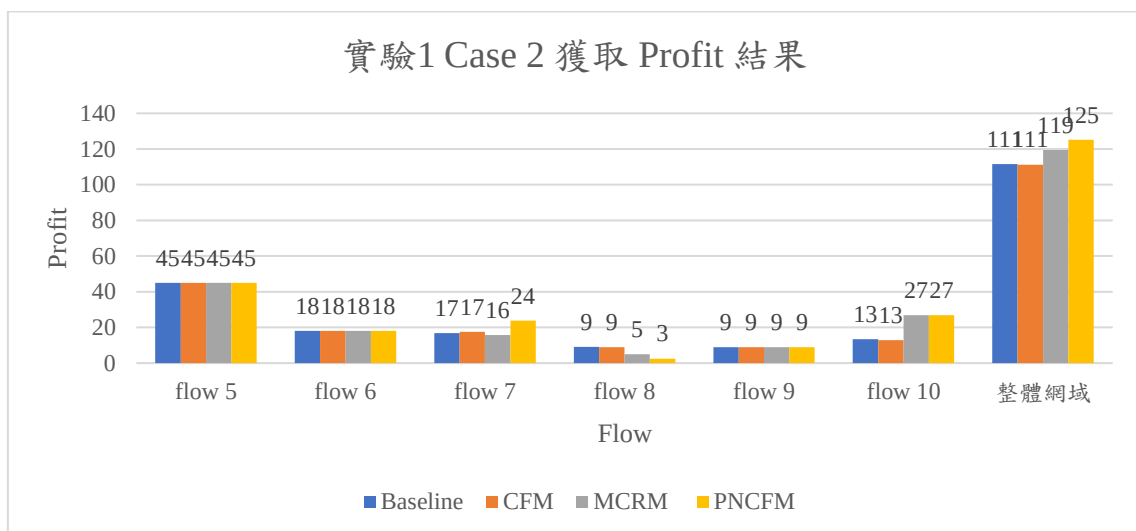


圖 6-11：實驗 1 Case 2 所獲取的 Profit

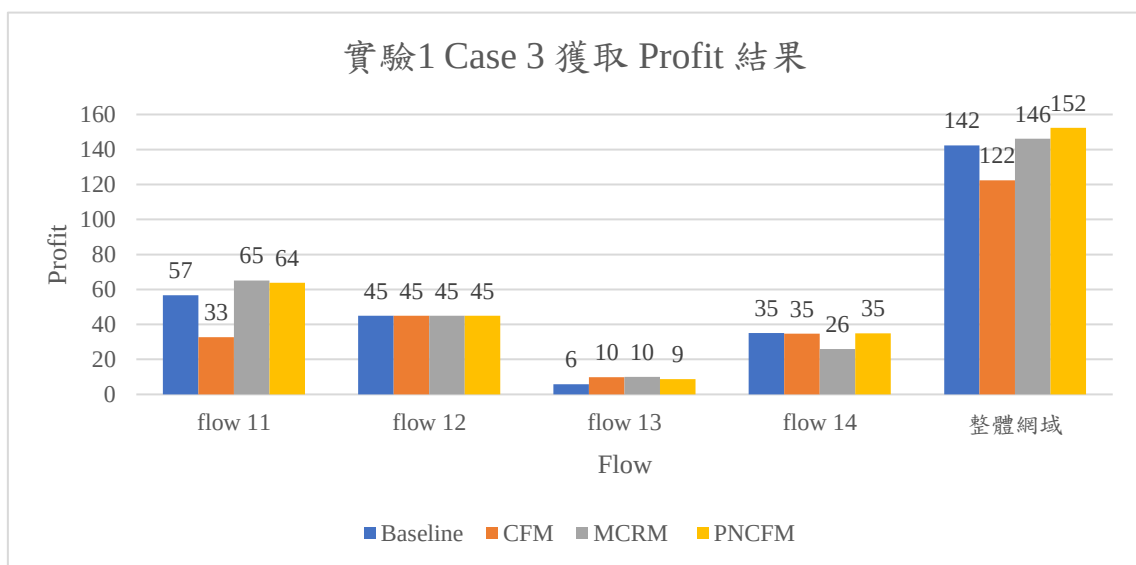


圖 6-12：實驗 1 Case 3 所獲取的 Profit

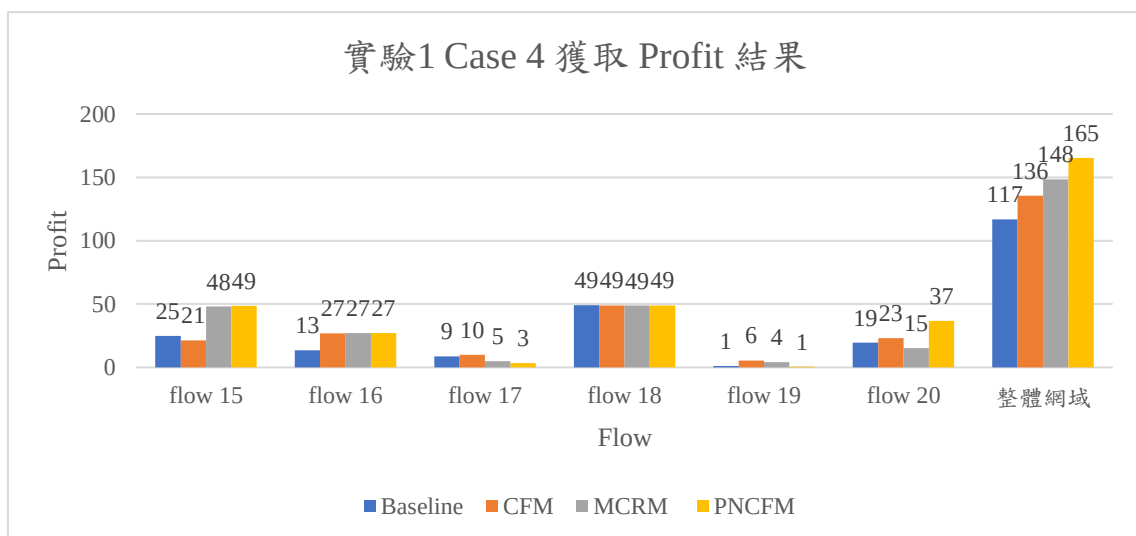


圖 6-13：實驗 1 Case 4 所獲取的 Profit

| Cases  | Flow    | Profit | Unit Profit | Baseline<br>封包丟失率 | CFM<br>封包丟失率 | MCRM<br>封包丟失率 | PNCFM<br>封包丟失率 | Baseline<br>獲取 Profit | CFM<br>獲取 Profit | MCRM<br>獲取 Profit | PNCFM<br>獲取 Profit |
|--------|---------|--------|-------------|-------------------|--------------|---------------|----------------|-----------------------|------------------|-------------------|--------------------|
| Case 1 | flow 1  | 63     | 7           | 66.49%            | 0.28%        | 0.25%         | 0.39%          | 21                    | 63               | 63                | 63                 |
|        | flow 2  | 18     | 2           | 13.25%            | 2.43%        | 2.52%         | 3.41%          | 16                    | 18               | 18                | 17                 |
|        | flow 3  | 21     | 3           | 0.00%             | 0.00%        | 0.00%         | 0.00%          | 21                    | 21               | 21                | 21                 |
|        | flow 4  | 30     | 10          | 0.00%             | 0.00%        | 0.00%         | 0.00%          | 30                    | 30               | 30                | 30                 |
|        | 整體網域    |        |             | 26.36%            | 0.93%        | 0.96%         | 1.31%          | 88                    | 131              | 131               | 131                |
| Case 2 | flow 5  | 45     | 5           | 0.01%             | 0.00%        | 0.01%         | 0.01%          | 45                    | 45               | 45                | 45                 |
|        | flow 6  | 18     | 2           | 0.01%             | 0.01%        | 0.01%         | 0.01%          | 18                    | 18               | 18                | 18                 |
|        | flow 7  | 24     | 4           | 29.64%            | 27.28%       | 34.41%        | 0.91%          | 17                    | 17               | 16                | 24                 |
|        | flow 8  | 14     | 2           | 34.66%            | 36.10%       | 65.11%        | 82.07%         | 9                     | 9                | 5                 | 3                  |
|        | flow 9  | 9      | 3           | 0.00%             | 0.00%        | 0.00%         | 0.00%          | 9                     | 9                | 9                 | 9                  |
|        | flow 10 | 30     | 10          | 55.25%            | 57.17%       | 10.70%        | 10.37%         | 13                    | 13               | 27                | 27                 |
|        | 整體網域    |        |             | 15.29%            | 15.29%       | 17.89%        | 15.20%         | 111                   | 111              | 119               | 125                |
| Case 3 | flow 11 | 72     | 8           | 21.37%            | 54.45%       | 9.49%         | 11.42%         | 57                    | 33               | 65                | 64                 |
|        | flow 12 | 45     | 5           | 0.00%             | 0.01%        | 0.02%         | 0.00%          | 45                    | 45               | 45                | 45                 |
|        | flow 13 | 10     | 2           | 42.36%            | 1.01%        | 0.05%         | 13.31%         | 6                     | 10               | 10                | 9                  |
|        | flow 14 | 35     | 5           | 0.00%             | 1.02%        | 25.98%        | 0.40%          | 35                    | 35               | 26                | 35                 |
|        | 整體網域    |        |             | 14.04%            | 17.90%       | 8.16%         | 6.01%          | 142                   | 122              | 146               | 152                |
| Case 4 | flow 15 | 56     | 7           | 55.46%            | 62.05%       | 14.19%        | 13.16%         | 25                    | 21               | 48                | 49                 |
|        | flow 16 | 27     | 3           | 50.02%            | 0.06%        | 0.02%         | 0.01%          | 13                    | 27               | 27                | 27                 |
|        | flow 17 | 12     | 2           | 27.35%            | 17.52%       | 58.36%        | 70.95%         | 9                     | 10               | 5                 | 3                  |
|        | flow 18 | 49     | 7           | 0.00%             | 0.15%        | 0.26%         | 0.11%          | 49                    | 49               | 49                | 49                 |
|        | flow 19 | 6      | 2           | 80.48%            | 8.15%        | 31.49%        | 88.60%         | 1                     | 6                | 4                 | 1                  |
|        | flow 20 | 50     | 10          | 61.05%            | 53.97%       | 69.41%        | 26.73%         | 19                    | 23               | 15                | 37                 |
|        | 整體網域    |        |             | 43.54%            | 23.42%       | 23.00%        | 24.04%         | 117                   | 136              | 148               | 165                |

表 6-3：實驗 1 封包丟失率與 Profit 獲取

| Cases  | Baseline<br>獲得 Profit | CFM 獲得 Profit | MCRM 獲得 Profit | PNCFM 獲得 Profit | PNCFM vs Baseline | PNCFM vs CFM | PNCFM vs MCRM |
|--------|-----------------------|---------------|----------------|-----------------|-------------------|--------------|---------------|
| Case 1 | 87.73                 | 131.39        | 131.39         | 131.14          | 49%               | 0%           | 0%            |
| Case 2 | 111.45                | 111.25        | 119.41         | 125.17          | 12%               | 13%          | 5%            |
| Case 3 | 142.37                | 122.33        | 146.06         | 152.31          | 7%                | 25%          | 4%            |
| Case 4 | 116.80                | 135.59        | 148.32         | 165.37          | 42%               | 22%          | 11%           |

表 6-4：實驗 1 各情境中 Profit 獲取以及各方法與 PNCFM 結果之比較



### 6.3.3 封包丟失率分析

封包丟失常因鏈路壅塞、資源分配不均或路由策略不當所導致，進而使得資料傳輸失敗，降低整體系統效益與服務品質，因此，本章節將針對各方法（包含 Baseline、CFM、MCRM 及本論文所提出之 PNCFM）於不同實驗情境下的封包丟失率進行分析與比較。由表 6-3 可觀察，各實驗案例中，PNCFM 在封包丟失率的表現皆優於其他機制或是與其他方式相近，以情境 1 來說，Baseline 封包丟失率高達 26.36%，至於其他方式可透過跨域合作舒緩網域壅塞將封包丟失率顯著降低至 1% 附近，其改善幅度達 25% 以上。此結果顯示，在傳統未進行流量協調與跨域合作的架構下，封包極易因資源競爭而遭丟棄，而透過跨域合作機制，能有效緩解此問題。

進一步觀察情境 2，PNCFM、MCRM、CFM、Baseline 的表現分別為 15.20%、17.89%、15.29%、15.29%，顯示在網域壅塞且無其他網域可以進行協助的情況下，整體的傳送達到了一個上限，所以 PNCFM、CFM、Baseline 表現相近，而 MCRM 因為自身的頻寬分配方式沒有完整分配頻寬，導致其封包丟失率上升。

在情境 3 中，原網域壅塞使得 CFM、MCRM、PNCFM 會進行跨域合作以藉由協助網域來舒緩原網域的壅塞，而後續協助網域也發生了壅塞，此時 MCRM 與 PNCFM 會拆分資料流，部分透過原網域傳輸到目的地，部分則會透過協助網域，也因此，PNCFM 與 MCRM 的表現會比 CFM 與 Baseline 來得好，特別來說，在整體網域的封包丟失率上，PNCFM、MCRM、CFM、Baseline 的表現分別為 6.01%、8.16%、17.90%、14.04%，此處 CFM 因為把高頻寬的資料流（也就是 flow 11）導到協助網域後卻沒有在壅塞時有其他處理，所以封包丟失率反而比什麼都不做的 Baseline 方法來的差，至於 MCRM 因為會用 Profit 比例來決定分配給協助流的頻寬，進而犧牲部分協助網域的原始資料流，所以提升封包丟失率，相較

之下 PNCFM 中，協助流提供的 Profit 因為不夠使協助網域釋放出非空間的頻寬，因此沒有犧牲協助網域原始資料流的頻寬，所以封包丟失率較低。

接下來，我們觀察情境 4，其狀況與情境 2 類似，網域 1 的部分鏈路形成壅塞，因而啟動部分方法的壅塞管理機制，不同之處為部分資料流有透過協助網域傳輸，但是後續其他資料流啟動後原網域與協助網域壅塞，協助網域無法進行更多協助，所以有跨域合作的方式要在壅塞處自行進行頻寬分配，這包含 CFM、MCRM、PNCFM，而其中 CFM 沒有另外設計頻寬分配的機制，實驗結果顯示三個有跨域合作機制的方式之封包丟失率在 23%至 24%，而沒有跨域合作機制的模式的封包丟失率為 43%，顯示有跨域合作能夠有效的降低頻寬丟失率。

## 6.4 實驗 2：不同 Profit 與頻寬需求的相關性分析

接著，我們要對 Profit 與頻寬需求相關性對於 Profit 獲取影響進行分析，特別來講，我們在實驗 1 情境 4 的基礎上，於壅塞情況下分為三個情境，正相關、負相關、不相關，比較的方法為 Baseline、CFM、MCRM、PNCFM，詳細測試資料流資訊可參考表 6-5，以下對測試的三個情境進行說明：

1. Profit 與頻寬需求正相關：頻寬需求越高，資料流 Profit 也就越高，此情境之資料流存續時間軸可參考圖 6-14。
2. Profit 與頻寬需求負相關：頻寬需求越高，資料流 Profit 也就越低，此情境之資料流存續時間軸可參考圖 6-15。
3. Profit 與頻寬需求不相關：頻寬需求越高，不代表資料流 Profit 也會越高，是個正相關負相關情都有的情境，此情境之資料流存續的時間軸可參考圖 6-16。

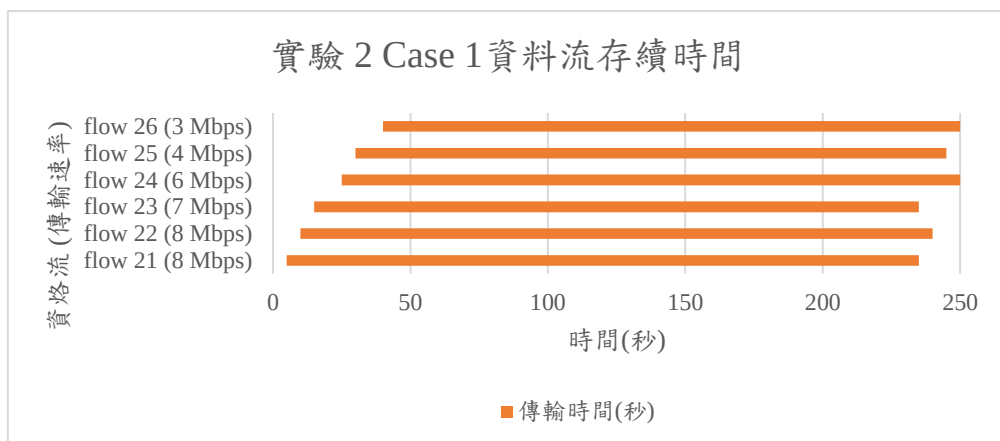


圖 6-14：實驗 2 Case 1 資料流存續時間

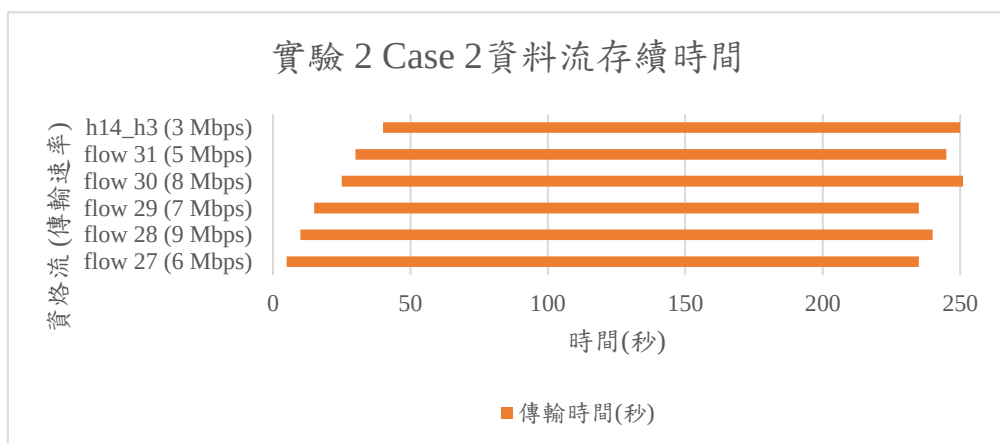


圖 6-15：實驗 2 Case 2 資料流存續時間

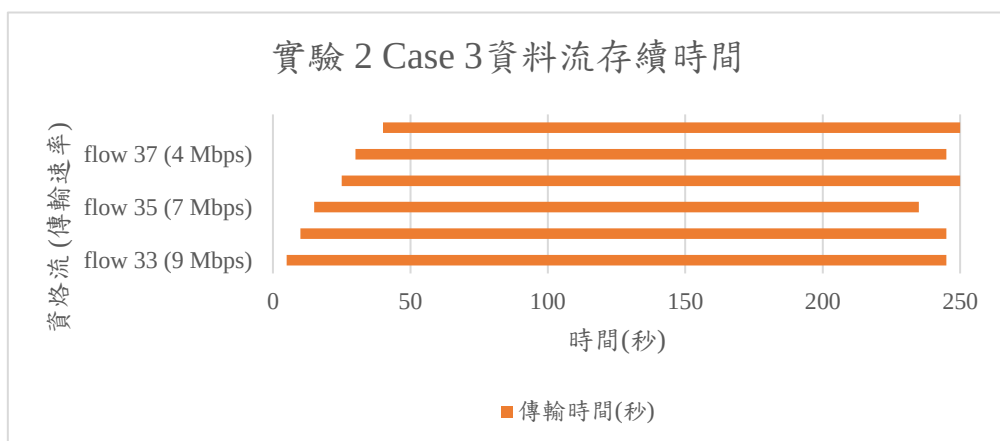


圖 6-16：實驗 2 Case 3 資料流存續時間

| Case   | 起始主機 | 目標主機 | 資料流     | 頻寬 (Mbps) | 單位 Profit | 總 Profit | 起始時間 | 結束秒數 | 傳輸時間 | 跨網域 |
|--------|------|------|---------|-----------|-----------|----------|------|------|------|-----|
| Case 1 | h1   | h6   | flow 21 | 8         | 11        | 88       | 5    | 235  | 230  | ✓   |
|        | h9   | h10  | flow 22 | 8         | 11        | 88       | 10   | 240  | 230  |     |
|        | h2   | h3   | flow 23 | 7         | 8         | 56       | 15   | 235  | 220  |     |
|        | h7   | h13  | flow 24 | 6         | 8         | 48       | 25   | 250  | 225  |     |
|        | h5   | h4   | flow 25 | 4         | 5         | 20       | 30   | 245  | 215  |     |
|        | h14  | h3   | flow 26 | 3         | 3         | 9        | 40   | 250  | 210  |     |
| Case 2 | h1   | h6   | flow 27 | 6         | 4         | 24       | 5    | 235  | 230  | ✓   |
|        | h9   | h10  | flow 28 | 9         | 1         | 9        | 10   | 240  | 230  |     |
|        | h2   | h3   | flow 29 | 7         | 3         | 21       | 15   | 235  | 220  |     |
|        | h7   | h13  | flow 30 | 8         | 2         | 16       | 25   | 255  | 230  |     |
|        | h5   | h4   | flow 31 | 5         | 6         | 30       | 30   | 245  | 215  |     |
|        | h14  | h3   | flow 32 | 3         | 13        | 39       | 40   | 250  | 210  |     |
| Case 3 | h1   | h6   | flow 33 | 9         | 12        | 108      | 5    | 245  | 240  | ✓   |
|        | h9   | h10  | flow 34 | 8         | 3         | 24       | 10   | 245  | 235  |     |
|        | h2   | h3   | flow 35 | 7         | 4         | 28       | 15   | 235  | 220  |     |
|        | h7   | h13  | flow 36 | 6         | 9         | 54       | 25   | 250  | 225  |     |
|        | h5   | h4   | flow 37 | 4         | 2         | 8        | 30   | 245  | 215  |     |
|        | h14  | h3   | flow 38 | 5         | 12        | 60       | 40   | 250  | 210  |     |

表 6-5：實驗 2 各情境的資料流資訊

## 6.4.1 實驗 2 結果分析

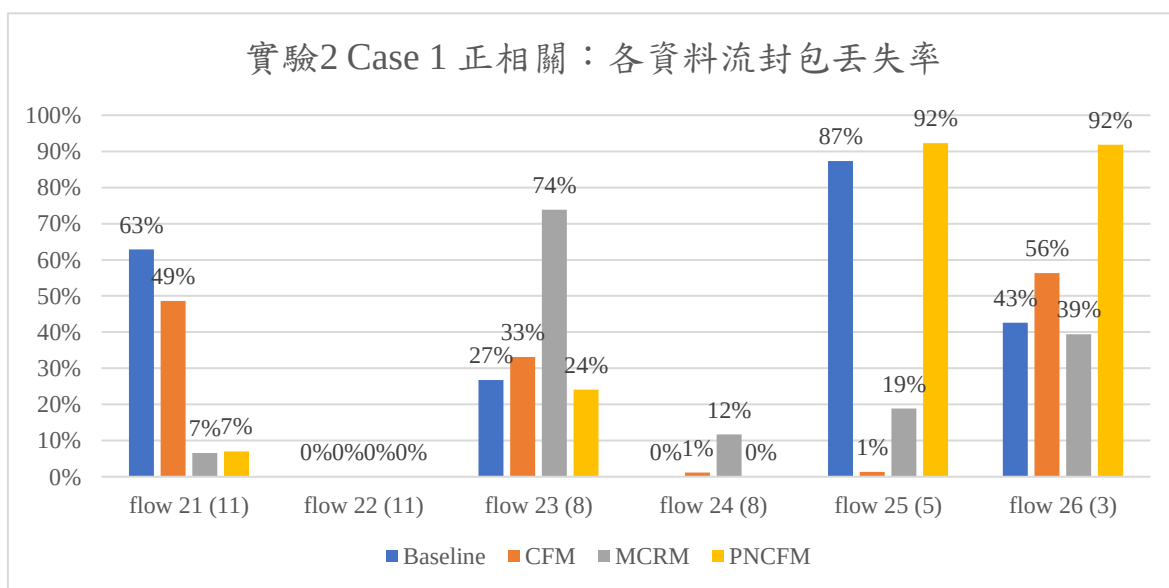


圖 6-17：實驗 2 Case 1 各資料流封包丟失率

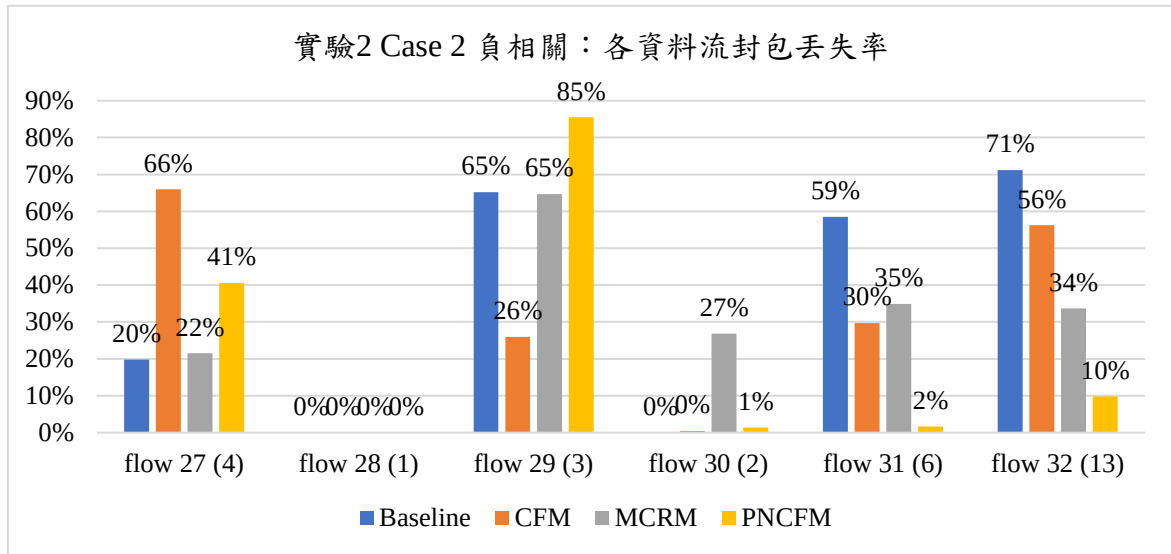


圖 6-18：實驗 2 Case 2 各資料流封包丟失率

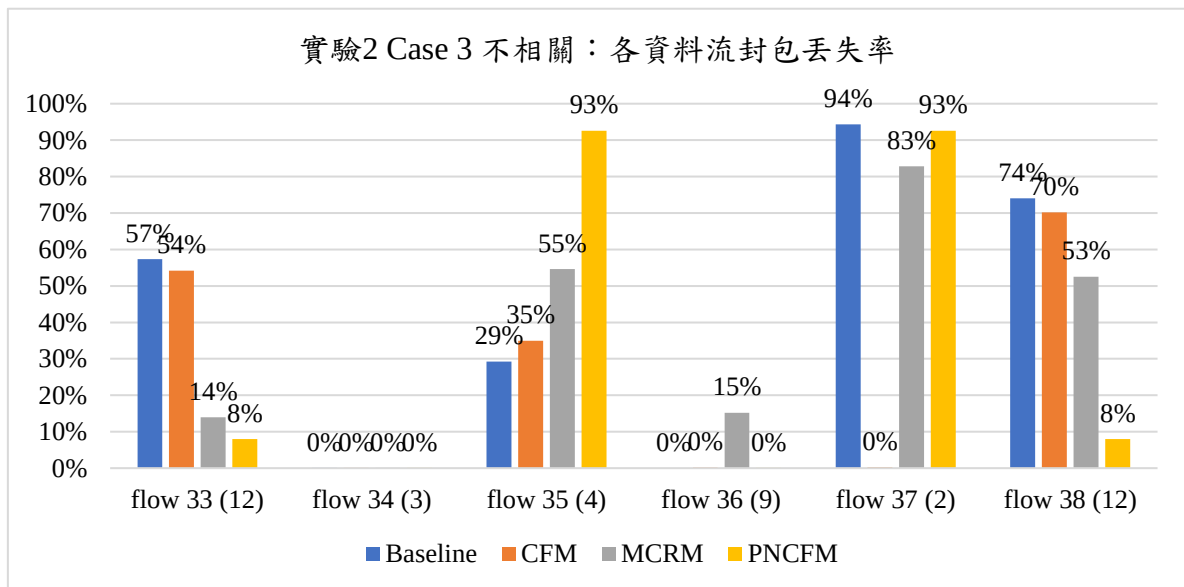


圖 6-19：實驗 2 Case 3 各資料流封包丟失率

| Cases  | Flow    | Profit | Unit Profit | Baseline<br>封包丟失率 | CFM<br>封包丟失率 | MCRM<br>封包丟失率 | PNCFM<br>封包丟失率 | Baseline<br>Profit 獲取 | CFM<br>Profit 獲取 | MCRM<br>Profit 獲取 | PNCFM<br>Profit 獲取 |
|--------|---------|--------|-------------|-------------------|--------------|---------------|----------------|-----------------------|------------------|-------------------|--------------------|
| Case 1 | flow 21 | 88     | 11          | 63%               | 49%          | 7%            | 7%             | 32.69                 | 45.23            | 82.21             | 81.81              |
|        | flow 22 | 88     | 11          | 0%                | 0%           | 0%            | 0%             | 88.00                 | 87.99            | 87.99             | 87.99              |
|        | flow 23 | 56     | 8           | 27%               | 33%          | 74%           | 24%            | 41.02                 | 37.45            | 14.64             | 42.52              |
|        | flow 24 | 48     | 8           | 0%                | 1%           | 12%           | 0%             | 48.00                 | 47.44            | 42.39             | 47.99              |
|        | flow 25 | 20     | 5           | 87%               | 1%           | 19%           | 92%            | 2.53                  | 19.73            | 16.23             | 1.55               |
|        | flow 26 | 9      | 3           | 43%               | 56%          | 39%           | 92%            | 5.17                  | 3.93             | 5.45              | 0.73               |
|        | 整體網域    |        |             | 32%               | 22%          | 23%           | 23%            | 217.40                | 241.78           | 248.91            | 262.59             |
| Case 2 | flow 27 | 24     | 4           | 20%               | 66%          | 22%           | 41%            | 19.25                 | 8.16             | 18.83             | 14.27              |
|        | flow 28 | 9      | 1           | 0%                | 0%           | 0%            | 0%             | 9.00                  | 9.00             | 8.99              | 9.00               |
|        | flow 29 | 21     | 3           | 65%               | 26%          | 65%           | 85%            | 7.30                  | 15.55            | 7.42              | 3.05               |
|        | flow 30 | 16     | 2           | 0%                | 0%           | 27%           | 1%             | 16.00                 | 15.92            | 11.71             | 15.77              |
|        | flow 31 | 30     | 6           | 59%               | 30%          | 35%           | 2%             | 12.45                 | 21.08            | 19.52             | 29.51              |
|        | flow 32 | 39     | 13          | 71%               | 56%          | 34%           | 10%            | 11.24                 | 17.08            | 25.86             | 35.18              |
|        | 整體網域    |        |             | 28%               | 23%          | 28%           | 24%            | 75.23                 | 86.79            | 92.34             | 106.78             |
| Case 3 | flow 33 | 108    | 12          | 57%               | 54%          | 14%           | 8%             | 46.11                 | 49.51            | 92.95             | 99.39              |
|        | flow 34 | 24     | 3           | 0%                | 0%           | 0%            | 0%             | 24.00                 | 24.00            | 24.00             | 24.00              |
|        | flow 35 | 28     | 4           | 29%               | 35%          | 55%           | 93%            | 19.81                 | 18.22            | 12.72             | 2.07               |
|        | flow 36 | 54     | 9           | 0%                | 0%           | 15%           | 0%             | 54.00                 | 53.92            | 45.81             | 53.99              |
|        | flow 37 | 8      | 2           | 94%               | 0%           | 83%           | 93%            | 0.46                  | 7.98             | 1.38              | 0.60               |
|        | flow 38 | 60     | 12          | 74%               | 70%          | 53%           | 8%             | 15.57                 | 17.90            | 28.49             | 55.21              |
|        | 整體網域    |        |             | 37%               | 28%          | 30%           | 28%            | 159.94                | 171.53           | 205.34            | 235.25             |

表 6-6：實驗 2 各情境下之封包丟失率與 Profit 獲取

| Cases      | Baseline<br>獲得 Profit | CFM 獲得 Profit | MCRM 獲得 Profit | PNCFM 獲得 Profit | PNCFM vs Baseline | PNCFM vs CFM | PNCFM vs MCRM |
|------------|-----------------------|---------------|----------------|-----------------|-------------------|--------------|---------------|
| Case 1 正相關 | 217.40                | 241.78        | 248.91         | 262.59          | 21%               | 9%           | 5%            |
| Case 2 負相關 | 75.23                 | 86.79         | 92.34          | 106.78          | 42%               | 23%          | 16%           |
| Case 3 Mix | 159.94                | 171.53        | 205.34         | 235.25          | 47%               | 37%          | 15%           |

表 6-7：實驗 2 各情境下總 Profit 獲取結果與 PNCFM 比其他方法改善幅度

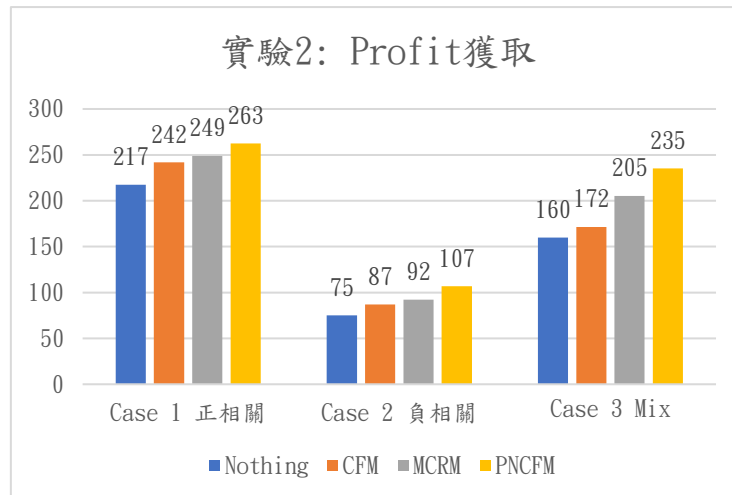


圖 6-20：實驗 2 各情境之總 Profit 獲取

各情境的 Profit 獲取結果如表 6-7 與圖 6-20 所示，可以發現本研究所提出的 PNCFM 方法在所有情境都可以在一定程度上表現得更好，當壅塞節點存在頻寬大小與價值多少存在反相關的情況下，PNCFM 相較於其他方式最少能提升 15% 的 Profit 獲取，部分反相關得情況下甚至能提升到 15% 的 Profit 獲取，而在正相關的情況下最少能提升 5% 的 Profit 獲取，由以上結果我們可以發現在資料流頻寬大小與 Profit 多少成正比時，獲得 Profit 之能力的提升會相對有限，而在情境二與情境三中我們可以發現當資料流有反比關係時，也就是當部分資料流指佔據少數頻寬，但是能夠獲得大量價值的情況下，本研究提出的 PNCFM 方法能夠有效分配頻寬來提升 Profit 的獲取。

接下來，我們針對各情境中每條資料流的封包丟失率進行分析，在各情境中，通過最壅塞節點的資料流分別有 h1 到 h6 的資料流、h2 到 h3 的資料流、h5 到 h4 的資料流、與 h14 到 h3 的資料流，PNCFM 在其他網域無法進一步提供協助時會在進行壅塞控管來提升頻寬，而 MCRM 在請求支援無果後會進行壅塞控管按照資料流總價值的比例來分配頻寬，至於 CFM 與 Baseline 則不會進行壅塞控管。

在情境一的壅塞節點上，將資料流的單位價值排序從高到低分別為 flow 21 > flow 23 > flow 25 > flow 26，而從圖 6-17 可以發現資料流 flow 21 的單位價值最高，而 PNCFM 與 MCRM 中它的封包丟失率都為 7%，單位價值次高的資料流為 flow

23，PNCFM 中它的封包丟失率為 24 %，遠低於 MCRM 中的 77%，因為價值在一個價值導向的環境中多少代表著它的重要性，而且單位價值越高，系統若能妥善運用則可以更有效率地分配頻寬來提升價值獲取，結合此實驗中 PNCFM 總體獲得價值高於 MCRM 5%，實驗結果顯示 PNCFM 能夠更有效的分配頻寬且保障較重要的資料流的頻寬。

情境 2 (反相關)中，實驗 2 情境 2 詳細結果可參考表 6-6、各資料流封包丟失率可看圖 6-18，若將資料流依的單位價值排序從高到低分別為 flow 32 > flow 31 > flow 27 > flow 29，PNCFM 中他們的封包丟失率分別為 11%、2%、42%、84%，相較於 MCRM 的 34%、35%、65%、22%，可以顯示在反相關的情況下，PNCFM 優先將頻寬分配給高單位價值的資料流，能較為有效的分配頻寬並提升整體價值產出約達 15%，相較於先前方式中表現最好的 MCRM，PNCFM 將價值獲取從 92 提升到 106。

情境 3 中，正相關、反相關或是不相關都存在，實驗詳細結果可參考表 6-6、各資料流封包丟失率可看圖 6-19，而相似於情境 2 的分析，若將資料流依單位價值排序，由大到小分別為 flow 33 = flow 38 > flow 35 > flow 37，在 PNCFM 當中，它們的封包丟失率分別為 8%、8%、93%、93%，在 MCRM 中，它們的封包丟失率分別為 14%、53%、55%、83%，顯示了 PNCFM 可以優先將頻寬分配給高單位價值的資料流，較有效率的使用頻寬，從而獲得更高價值的頻寬分配。



## 第七章 結論與未來展望

### 7.1 結論

本研究針對多網域的 SDN 架構提出一套流量管理機制，以進行跨域負載平衡與頻寬共享，確保高 Profit 資料流在資源競爭下能被優先保障，同時也避免低 Profit 值資料流不會發生 Starvation，進而提升整體系統的效益表現，當單一網域內因路徑資源不足導致壅塞時，我們所提出的方法允許向其他網域借用頻寬，以彈性調度跨網域的協助流，然而，由於每個網域中同時包含本地資料流與外來協助流，頻寬分配便成為一項重要課題，針對此點，我們利用 OpenFlow 的 Meter table 機制來限制協助流的最大傳輸速率，以避免其搶佔過多資源，並透過 Group table 實現多路徑的流量導引，將超額流量透過來源網域進行處理，以善用整體網路資源，並維持系統穩定傳輸，透過各網域控制器間的協同通訊與資源共享，我們可達成動態調度與擁塞疏解的效果，經由 Mininet 模擬驗證，本論文所提出的方法不僅能有效提升網路的整體吞吐能力，亦可在壅塞情況下提升系統 Profit 產出。

### 7.2 未來展望

在未來的研究方向中，我們希望在選擇資料流的決策可以考量各網域間的公平性，以及在單一網域內的各資料流的公平性，此外，在跨網域的協助傳輸時，我們尚未無考慮個別控制器負載以及可能發生故障等情況，藉未來也會針對避免單一弱點以及考量各控制器的負載平衡，最後，本實驗中尚未考慮將資料流進行排程，或是考量資料流應用性質，有可能部分應用不是時間敏感 (Time-sensitive) 的任務，也有可能資料流需要完整的頻寬才能對使用者而言是有用的，只獲得一部分就跟沒有一樣，例如直播(livestream)，而這樣不考慮應用場景有可能降低使用者體驗，或是在系統壅塞時傳送不需要當下完成的封包而增加鏈路壅塞，於此情況下，我們可以考慮將部分資料流進行排程，並於非尖峰時段進行傳輸。

## 參考文獻

- [1] C. Urrea and D. Benítez, “Software-defined networking solutions, architecture and controllers for the industrial Internet of Things: A review,” *Sensors*, vol. 21, no. 19, pp. 1-20, 2021.
- [2] R. Aryan, A. Yazidi, F. Brattensborg, Ø. Kure, and P. E. Engelstad, “SDN spotlight: A real-time OpenFlow troubleshooting framework, ” *Future Generation Computer Systems*, vol. 133, pp. 364-377, 2022.
- [3] Y. C. Wang and L. C. Yen, “Collaborative route management to mitigate congestion in multi-domain networks using SDN, ” *IEEE Computing and Communication Workshop and Conference*, 2022, pp. 988-994.
- [4] Y. C. Wang and J. T. Liang, “POFM: Profit-oriented flow management in SDN-based data center networks,” *IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology*, 2024, pp. 130-135.
- [5] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, “OpenFlow: Enabling innovation in campus networks, ” *SIGCOMM Computer Communication Review*, pp. 69–74, 2008.
- [6] OpenFlow.1.5.1 [Online] Available: <https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf>
- [7] Beacon Controller [Online] Available: <https://github.com/bigswitch/BeaconMirror>
- [8] OpenDaylight [Online] Available: <https://www.opendaylight.org/>
- [9] ONOS [Online] Available: <https://onosproject.org/>
- [10] Ryu SDN Framework. [Online] Available: <https://osrg.github.io/ryu->

- [11] A. Tootoonchian and Y. Ganjali, "HyperFlow: A distributed control plane for OpenFlow," *Internet Network Management Conference on Research on Enterprise Networking*, 2010, pp.1-6.
- [12] M. Priyadarsini and P. Bera, "Software defined networking architecture, traffic management, security, and placement: A survey" , *Computer Networks*, vol.192, 2021, pp. 1-15.
- [13] M. N. A. Sheikh, I. S. Hwang, M. S. Raza, and M. S. Ab-Rahman, "A qualitative and comparative performance assessment of logically centralized SDN controllers via mininet emulator," *Computers*, Vol. 13, no. 4, pp. 1-27, 2024.
- [14] B. B. Deneke, A. M. Beyene, and E. A. Haile, "Improving software defined network controllers in a multi-vendor environment," *Heliyon*, vol. 10, no. 4, pp. 1-13, 2024.
- [15] K. Yalda, D. J. Hamad, and N. Tapus, "Comparative analysis of centralized and distributed SDN environments for IoT networks," *Control Engineering and Applied Informatics*, vol. 26, pp. 84-91, 2024.
- [16] H. G. Ahmed and R. Ramalakshmi, "Performance analysis of centralized and distributed SDN controllers for load balancing application," *International Conference on Trends in Electronics and Informatics*, 2018, pp. 758-764.
- [17] M. Rahouti, K. Xiong, Y. Xin, and N. Ghani, "QoSP: A priority-based queueing mechanism in software-defined networking environments," *IEEE International Performance, Computing, and Communications Conference*, 2021, pp. 1-7.
- [18] M. Karakus and A. Durresi, "A survey: Control plane scalability issues and approaches in software-defined networking (SDN)," *Computer Networks*, vol. 112, pp. 279-293, 2017.

- [19] T. Koponen, M. Casado, N. Gude, J. Stribling, L. Poutievski, M. Zhu, R. Ramanathan, Y. Iwata, H. Inoue, T. Hama, and S. Shenker, "Onix: A distributed control platform for large-scale production networks," *USENIX Conference on Operating Systems Design and Implementation*, 2010, pp. 351–364.
- [20] T. Zhang, A. Bianco, and P. Giaccone, "The role of inter-controller traffic in SDN controllers placement," *IEEE Conference on Network Function Virtualization and Software Defined Networks*, 2016, pp. 87-92.
- [21] F. Benamrane, M. B. Mamoun, and R. Benaini, "An east-west interface for distributed SDN control plane: Implementation and evaluation," *Computers & Electrical Engineering*, vol. 57, pp. 162-175, 2017.
- [22] J. Ali and B. H. Roh, "An efficient approach for load balancing in software-defined networks," *International Conference on Computing, Communication and Security*, 2021, pp. 1-6.
- [23] S. Li, Z. Xin, X. Xu, and Z. Zhang, "Load balancing algorithm of SDN controller based on dynamic threshold," *Asia-Pacific Conference on Communications Technology and Computer Science*, 2023, pp. 517-520.
- [24] W. H. F. Aly, "Controller adaptive load balancing for SDN networks," *International Conference on Ubiquitous and Future Networks*, 2019, pp. 514-519.
- [25] H. Mokhtar, X. Di, Y. Zhou, A. Hassan, Z. Ma, and S. Musa, "Multiple-level threshold load balancing in distributed SDN controllers," *Computer Networks*, 2021, vol. 198, pp 1-16,
- [26] S. Patil, "Load balancing approach for finding best path in SDN," *International Conference on Inventive Research in Computing Applications*, 2018, pp. 612-616.
- [27] B. Le and D. T. T. Van, "Load balancing routing under constraints of quality of

- transmission in mesh wireless network based on software-defined networking,” *Journal of Communications and Networks*, vol. 23, pp. 12-22, 2021.
- [28] T. S. Andjamba and G. A. L. Zodi, “A load balancing protocol for improved video on demand in SDN-based clouds,” *International Conference on Ubiquitous Information Management and Communication*, 2023, pp.1-6.
  - [29] V. Akella and K. Xiong, “Quality of service (QoS)-guaranteed network resource allocation via software defined networking (SDN),” *IEEE International Conference on Dependable, Autonomic and Secure Computing*, 2014, pp. 7-13.
  - [30] Ryu Documentation Release 4.34 [Online]. Available: [https://ryu.readthedocs.io/\\_/downloads/en/latest/pdf/](https://ryu.readthedocs.io/_/downloads/en/latest/pdf/)
  - [31] SciPy v1.11.1 Manual: `scipy.optimize.minimize` [Online] Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html>
  - [32] E. Sadeghi, H. Behroozi, and S. Rini, “Fairness-oriented user association in HetNets using bargaining game theory” , arXiv:2011.04801, 2020, pp.1-8.
  - [33] D. Liu, Y. Chen, K. K. Chai, and T. Zhang, “Nash bargaining solution based user association optimization in HetNets,” *IEEE Consumer Communications and Networking Conference*, 2014, pp. 587-592.
  - [34] SciPy v1.16.0 Manual: `scipy.optimize.minimize_scalar` [Online] Available: [https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize\\_scalar.html#scipy.optimize.minimize\\_scalar](https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize_scalar.html#scipy.optimize.minimize_scalar)
  - [35] Mininet. [Online]. Available: <http://mininet.org/>
  - [36] Stochastic Switching. [Online]. Available : <https://github.com/saeenali/openvswitch/wiki/Stochastic-Switching-using-Open-vSwitch-in-Mininet>
  - [37] Linux manual page: `tc-tbf(8)` [Online] Available: <https://man7.org/linux/man->

pages/man8/tc-tbf.8.html

- [38] Y. C. Wang and E. J. Chang, “Cooperative Flow Management in Multi-domain SDN-based Networks with Multiple Controllers, ” *IEEE International Conference on Smart Communities: Improving Quality of Life Using ICT, IoT and AI*, 2020, pp. 82-86.
- [39] B. Kline, “ Identification of complete information games,” *Journal of Econometrics*, 2015, pp.117-131.
- [40] Silva, W.J.A. “ An architecture to manage oncoming traffic of inter-domain routing using OpenFlow networks,” *Information* 2018, pp. 1-23.
- [41] P. Podili, K. Kataoka, “TRAQR: Trust aware end-to-end QoS routing in multi-domain SDN using blockchain,” *Journal of Network and Computer Applications*, Volume 182 , 2021, pp. 1-19.
- [42] Stewart, James. “Calculus: Early transcendentals” *Cengage Learning*, 2015, 8th ed, pp. 287-324.

# Appendix

本研究所建構的 Nash Bargaining 模組屬於完全資訊博弈 (Complete Information Game)，雙方玩家皆清楚彼此的效用函數、分歧點與可行集合，因此可直接運用 Nash 談判解進行分析，以下針對 Nash Bargaining 模組是否符合 Nash Bargaining 的四個公理進行論證，在此之前先列出本模組的相關公式。

1. 本模組中建立的賽局：

$$U^* = \arg \max_u \prod_{n=1}^N (u_n - d_n) \quad , s.t. \ u_n \geq d_n \quad (14)$$

2. 玩家 1 的功能函數，其代表玩家 1 在合作後能增加多少 Profit 價值：

$$u_1 = (1 - s) \times \frac{B_1^a}{B_1^d} \times \bar{P}_1 \quad , \quad (15)$$

$$s.t. \ 0 \leq B_1^a \leq B_1^d \quad , \ 0 \leq s \leq 1$$

3. 玩家 1 的分歧點：

$$d_1 = 0 \quad (16)$$

4. 玩家 2 的功能函數，其代表玩家 2 在合作後能增加多少 Profit 價值：

$$u_2 = \bar{P}_2 \times \frac{\min(C_l - B_1^a, B_2^d)}{B_2^d} + (s) \times \frac{B_1^a}{B_1^d} - \bar{P}_2 \times \min\left(\frac{C_l}{B_2^d}, 1\right) \quad , \quad (17)$$

$$s.t. \ 0 \leq B_1^a \leq B_1^d \quad , \ 0 \leq s \leq 1$$

5. 玩家 2 的分歧點：

$$d_2 = 0 \quad (18)$$

6. 結合(3)、(5)、(6)、(7)、(8)，將公式帶入後的賽局：

$$U^* = \arg \max_{B_1^a} \left\{ \left[ (1 - s) \times \frac{B_1^a}{B_1^d} \times \bar{P}_1 \right] \right. \\ \left. \times \left[ \bar{P}_2 \times \frac{\min(C_l - B_1^a, B_2^d)}{B_2^d} + (s) \times \frac{B_1^a}{B_1^d} - \bar{P}_2 \times \min\left(\frac{C_l}{B_2^d}, 1\right) \right] \right\} \quad (19)$$

$$, s.t. \ 0 \leq B_1^a \leq B_1^d \quad , \ 0 \leq s \leq 1$$

接下來繼續對 Nash 四大公理的滿足性進行分析：

1. 效率性(Pareto Optimality) (即效率性、不受額外無關選項影響):

若找到的解不是 Pareto 最適解，表示還有另一個可行點能讓至少一方更好而另一方不變差，這樣兩者乘積一定會變大，所以原來就不是最大乘積。

以下採取反證法來證明：

假設 Nash Bargaining 模組所求得的最大化解 $(u_1^*, u_2^*)$ 並不位於 Pareto 效率邊界上。根據 Pareto 效率的定義，如果 $(u_1^*, u_2^*)$ 不在邊界上，那麼在可行的效用集中，必然存在另一個點 $(u_1', u_2')$ ，使得 $u_1' \geq u_1^*$ 且 $u_2' \geq u_2^*$ 並且至少其中一個是不等號。

然而，因為 $u_1' \geq u_1^*$ 和 $u_2' \geq u_2^*$ ，且 Nash Product 僅在效用高於不合作點時有意義，因此 $(u_1' - d_1)$ 和 $(u_2' - d_2)$ 都是正數。這將導致新的 Nash Product  $N' = (u_1' - d_1)(u_2' - d_2)$ 必定大於原始的 $N^* = (u_1^* - d_1)(u_2^* - d_2)$ 。

這個結果與我們一開始假設 $(u_1^*, u_2^*)$ 是最大化解的條件相矛盾。因此，我們的假設是錯誤的，Nash Bargaining Solution 的解必定位於 Pareto 效率邊界上。

## 2. 對稱性 (Symmetry)

對稱性公理要求若玩家地位相同，解也必須相同，但本研究情境下，協助流對協助網域的重要性較低，協助網域也沒有提供協助的義務，因此雙方在現實中權力本就不對等，同時，本模組中兩位玩家的效用函數形式不同，數學上並不具備互換性。由此可知，本模型不符合對稱性公理，然而，這並非缺陷，而是反映出此賽局的本質：一個不對稱且不公平的合作環境。

## 3. 獨立於不相關選項 (Independence of Irrelevant Alternatives, IIA)

IIA 公理要求，如果一個解決方案是某個可行集合的解，那麼它也必須是該集合中包含此解的任何子集的解，Nash Bargaining Solution 的解是透過最大化 Nash Product 找到的唯一點，假設 Nash Product 在可行集內，U 上找到最大值 $(u_1^*, u_2^*)$ 。現在，我們移除 U 中的一些無關因素，形成一個新的可行集 U'，其中 U' 是 U 的子集，且 $(u_1^*, u_2^*)$ 仍然在 U' 內，代表 $(u_1^*, u_2^*)$ 也是 U' 內的一個解，由於



$(u_1^*, u_2^*)$  在更大的集合  $U$  中已經是最大值，因此在更小的子集  $U'$  中，它也必然是最大值。

由於本模組採用 Nash Product 作為優化目標，其數學特性自然保證 IIA 定理的成立。

#### 4. 尺度不變性(Independence from affine utility transformation, IAT):

IAT 公理要求，如果我們對玩家的效用函數進行線性轉換，例如： $(u' = A \times u + B)$ ，最終的解決方案不變。

假設將模組內的 Nash Product 進行轉換： $(u'_1 = A \times u_1 + B)$  和  $(u'_2 = C \times u_2 + D)$ ，則  $U' = (Au_1 + B)(Cu_2 + D)$ ，由於  $A$  和  $C$  是正數常數，最大化  $U'$  的解與最大化原始的 Nash Product  $U$  的解完全相同，常數  $AC$  不會改變最大化點的位置。

總上所述，本研究所建構的 Nash Bargaining 模組屬於完全資訊博弈 (Complete Information Game)，雙方玩家皆清楚彼此的效用函數、分歧點與可行集合，因此可直接運用 Nash 談判解進行分析，在四項公理中，本模型於效率性 (Pareto efficiency)、獨立於不相關選項 (IIA)、以及尺度不變性 (Invariance) 上均能成立，確保解具有合理性與穩健性。然而，對稱性 (Symmetry) 公理並不適用，原因在於協助流與協助網域之間的現實差異：權力結構不均衡，且效用函數並不對稱。這種偏離反映真實網路環境下的不公平談判，而非理論上之缺陷。